Dokumentasi Resmi & Sintaks Rust (2021/2024 Edition Up-to-Dat

Bahasa: Rust | Total Chunk/Dokumen: 72 | Disiapkan untuk RAG System

[rust_001] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Installation

Sumber: ht

Installation
The first step is to install Rust. We'll download Rust through `rustup`, a
command line tool for managing Rust versions and associated tools. You'll need
an internet connection for the download.
> Note: If you prefer not to use `rustup` for some reason, please see the
> [Other Rust Installation Methods page][otherinstall] for more options.
The following steps install the latest stable version of the Rust compiler.
Rust's stability guarantees ensure that all the examples in the book that
compile will continue to compile with newer Rust versions. The output might
differ slightly between versions because Rust often improves error messages and
warnings. In other words, any newer, stable version of Rust you install using
these steps should work as expected with the content of this book.
> ### Command Line Notation
>
> In this chapter and throughout the book, we'll show some commands used in the
> terminal. Lines that you should enter in a terminal all start with `$`. You
> don't need to type the `$` character; it's the command line prompt shown to
> indicate the start of each command. Lines that don't start with `$` typically
> show the output of the previous command. Additionally, PowerShell-specific
> examples will use `>` rather than `$`.

[rust_002] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Installing `rustup` on Linux or macOS

Sumber: ht

Installing `rustup` on Linux or macOS
If you're using Linux or macOS, open a terminal and enter the following command:
The command downloads a script and starts the installation of the `rustup`
tool, which installs the latest stable version of Rust. You might be prompted
for your password. If the install is successful, the following line will appear:
You will also need a _linker_, which is a program that Rust uses to join its
compiled outputs into one file. It is likely you already have one. If you get
linker errors, you should install a C compiler, which will typically include a
linker. A C compiler is also useful because some common Rust packages depend on
C code and will need a C compiler.
On macOS, you can get a C compiler by running:
Linux users should generally install GCC or Clang, according to their
distribution's documentation. For example, if you use Ubuntu, you can install
the `build-essential` package.

Contoh Kode / Perintah CLI:

```
The command downloads a script and starts the installation of the `rustup`
tool, which installs the latest stable version of Rust. You might be prompted
for your password. If the install is successful, the following line will appear:
```

```
// ----------------------------------------

You will also need a _linker_, which is a program that Rust uses to join its
compiled outputs into one file. It is likely you already have one. If you get
linker errors, you should install a C compiler, which will typically include a
linker. A C compiler is also useful because some common Rust packages depend on
C code and will need a C compiler.

On macOS, you can get a C compiler by running:
```

[rust_003] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Installing `rustup` on Windows

Sumber: ht

Installing `rustup` on Windows
On Windows, go to [https://www.rust-lang.org/tools/install][install] and follow the instructions for installing Rust. At some point in the
installation, you'll be prompted to install Visual Studio. This provides a
linker and the native libraries needed to compile programs. If you need more
help with this step, see
[https://rust-lang.github.io/rustup/installation/windows-msvc.html][msvc].
The rest of this book uses commands that work in both _cmd.exe_ and PowerShell.
If there are specific differences, we'll explain which to use.

[rust_004] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Troubleshooting

Sumber: ht

Troubleshooting
To check whether you have Rust installed correctly, open a shell and enter this
line:
You should see the version number, commit hash, and commit date for the latest
stable version that has been released, in the following format:
If you see this information, you have installed Rust successfully! If you don't
see this information, check that Rust is in your `%PATH%` system variable as
follows.
In Windows CMD, use:
In PowerShell, use:
In Linux and macOS, use:
If that's all correct and Rust still isn't working, there are a number of
places you can get help. Find out how to get in touch with other Rustaceans (a
silly nickname we call ourselves) on [the community page][community].

Contoh Kode / Perintah CLI:

```
You should see the version number, commit hash, and commit date for the latest
stable version that has been released, in the following format:

// ----------------------------------------

If you see this information, you have installed Rust successfully! If you don't
see this information, check that Rust is in your `%PATH%` system variable as
follows.

In Windows CMD, use:

// ----------------------------------------
```

```
In PowerShell, use:
```

[rust_005] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Updating and Uninstalling

Sumber: ht

Updating and Uninstalling
Once Rust is installed via `rustup`, updating to a newly released version is easy. From your shell, run the following update script:
To uninstall Rust and `rustup`, run the following uninstall script from your shell:

Contoh Kode / Perintah CLI:

```
To uninstall Rust and `rustup`, run the following uninstall script from your
shell:
```

[rust_006] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Reading the Local Documentation

Sumber: ht

Reading the Local Documentation
The installation of Rust also includes a local copy of the documentation so that you can read it offline. Run `rustup doc` to open the local documentation in your browser.
Any time a type or function is provided by the standard library and you're not sure what it does or how to use it, use the application programming interface (API) documentation to find out!

[rust_007] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Using Text Editors and IDEs

Sumber: ht

Using Text Editors and IDEs
This book makes no assumptions about what tools you use to author Rust code. Just about any text editor will get the job done! However, many text editors and integrated development environments (IDEs) have built-in support for Rust. You can always find a fairly current list of many editors and IDEs on [the tools page][tools] on the Rust website.

[rust_008] Rust - Installation & Cargo Tooling

Installation and rustup Toolchain Setup: Working Offline with This Book

Sumber: ht

Working Offline with This Book
In several examples, we will use Rust packages beyond the standard library. To work through those examples, you will either need to have an internet connection or to have downloaded those dependencies ahead of time. To download the dependencies ahead of time, you can run the following commands. (We'll explain what `cargo` is and what each of these commands does in detail later.)
This will cache the downloads for these packages so you will not need to download them later. Once you have run this command, you do not need to keep the `get-dependencies` folder. If you have run this command, you can use the `--offline` flag with all `cargo` commands in the rest of the book to use these cached versions instead of attempting to use the network.

[otherinstall]: https://forge.rust-lang.org/infra/other-installation-methods.html
[install]: https://www.rust-lang.org/tools/install
[msvc]: https://rust-lang.github.io/rustup/installation/windows-msvc.html
[community]: https://www.rust-lang.org/community
[tools]: https://www.rust-lang.org/tools

[rust_009] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Hello, Cargo!

Sumber: ht

Hello, Cargo!
Cargo is Rust's build system and package manager. Most Rustaceans use this tool
to manage their Rust projects because Cargo handles a lot of tasks for you,
such as building your code, downloading the libraries your code depends on, and
building those libraries. (We call the libraries that your code needs
dependencies.)
The simplest Rust programs, like the one we've written so far, don't have any
dependencies. If we had built the "Hello, world!" project with Cargo, it would
only use the part of Cargo that handles building your code. As you write more
complex Rust programs, you'll add dependencies, and if you start a project
using Cargo, adding dependencies will be much easier to do.
Because the vast majority of Rust projects use Cargo, the rest of this book
assumes that you're using Cargo too. Cargo comes installed with Rust if you
used the official installers discussed in the
["Installation"][installation] section. If you installed Rust
through some other means, check whether Cargo is installed by entering the
following in your terminal:
If you see a version number, you have it! If you see an error, such as `command
not found`, look at the documentation for your method of installation to
determine how to install Cargo separately.

[rust_010] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Creating a Project with Cargo

Sumber: ht

Creating a Project with Cargo
Let's create a new project using Cargo and look at how it differs from our
original "Hello, world!" project. Navigate back to your _projects_ directory
(or wherever you decided to store your code). Then, on any operating system,
run the following:
The first command creates a new directory and project called _hello_cargo_.
We've named our project _hello_cargo_, and Cargo creates its files in a
directory of the same name.
Go into the _hello_cargo_ directory and list the files. You'll see that Cargo
has generated two files and one directory for us: a _Cargo.toml_ file and a
src directory with a _main.rs_ file inside.
It has also initialized a new Git repository along with a _.gitignore_ file.
Git files won't be generated if you run `cargo new` within an existing Git
repository; you can override this behavior by using `cargo new --vcs=git`.
> Note: Git is a common version control system. You can change `cargo new` to
> use a different version control system or no version control system by using

> the `--vcs` flag. Run `cargo new --help` to see the available options.
Open _Cargo.toml_ in your text editor of choice. It should look similar to the
code in Listing 1-2.
This file is in the [_TOML_][toml] (_Tom's Obvious, Minimal
Language_) format, which is Cargo's configuration format.
The first line, `[package]`, is a section heading that indicates that the
following statements are configuring a package. As we add more information to
this file, we'll add other sections.
The next three lines set the configuration information Cargo needs to compile
your program: the name, the version, and the edition of Rust to use. We'll talk
about the `edition` key in [Appendix E][appendix-e].
The last line, `[dependencies]`, is the start of a section for you

Contoh Kode / Perintah CLI:

```
The first command creates a new directory and project called _hello_cargo_.
We've named our project _hello_cargo_, and Cargo creates its files in a
directory of the same name.

Go into the _hello_cargo_ directory and list the files. You'll see that Cargo
has generated two files and one directory for us: a _Cargo.toml_ file and a
_src_ directory with a _main.rs_ file inside.

It has also initialized a new Git repository along with a _.gitignore_ file.
Git files won't be generated if you run `cargo new` within an existing Git
repository; you can override this behavior by using `cargo new --vcs=git`.

> Note: Git is a common version control system. You can change `cargo new` to
> use a different version control system or no version control system by using
> the `--vcs` flag. Run `cargo new --help` to see the available options.

Open _Cargo.toml_ in your text editor of choice. It should look similar to the
code in Listing 1-2.

<Listing number="1-2" file-name="Cargo.toml" caption="Contents of *Cargo.toml* generated by `cargo new`">

// ----------------------------------------

</Listing>

This file is in the [_TOML_][toml]<!-- ignore --> (_Tom's Obvious, Minimal
Language_) format, which is Cargo's configuration format.

The first line, `[package]`, is a section heading that indicates that the
following statements are configuring a package. As we add more information to
this file, we'll add other sections.

The next three lines set the configuration information Cargo needs to compile
your program: the name, the version, and the edition of Rust to use. We'll talk
about the `edition` key in [Appendix E][appendix-e]<!-- ignore -->.

The last line, `[dependencies]`, is the start of a section for you to list any
of your project's dependencies. In Rust, packages of code are referred to as
_crates_. We won't need any other crates for this project, but we will in the
first project in Chapter 2, so we'll use this dependencies section then.

Now open _src/main.rs_ and take a look:

<span class="filename">Filename: src/main.rs</span>
```

[rust_011] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Building and Running a Cargo Project

Sumber: ht

Building and Running a Cargo Project

Now let's look at what's different when we build and run the "Hello, world!" program with Cargo! From your _hello_cargo_ directory, build your project by entering the following command:

This command creates an executable file in _target/debug/hello_cargo_ (or _target\debug\hello_cargo.exe_ on Windows) rather than in your current directory. Because the default build is a debug build, Cargo puts the binary in a directory named _debug_. You can run the executable with this command:

If all goes well, `Hello, world!` should print to the terminal. Running `cargo build` for the first time also causes Cargo to create a new file at the top level: _Cargo.lock_. This file keeps track of the exact versions of dependencies in your project. This project doesn't have dependencies, so the file is a bit sparse. You won't ever need to change this file manually; Cargo manages its contents for you.

We just built a project with `cargo build` and ran it with `./target/debug/hello_cargo`, but we can also use `cargo run` to compile the code and then run the resultant executable all in one command:

Using `cargo run` is more convenient than having to remember to run `cargo build` and then use the whole path to the binary, so most developers use `cargo run`.

Notice that this time we didn't see output indicating that Cargo was compiling `hello_cargo`. Cargo figured out that the files hadn't changed, so it didn't rebuild but just ran the binary. If you had modified your source code, Cargo would have rebuilt the project before running it, and you would have seen this output:

Cargo also provides a command called `cargo check`. This command quickly checks your code to make sure it compiles but doesn't produce an executable:

Why would you not want an executable?

Contoh Kode / Perintah CLI:

```
This command creates an executable file in _target/debug/hello_cargo_ (or
_target\debug\hello_cargo.exe_ on Windows) rather than in your current
directory. Because the default build is a debug build, Cargo puts the binary in
a directory named _debug_. You can run the executable with this command:

// ----------------------------------------

If all goes well, `Hello, world!` should print to the terminal. Running `cargo
build` for the first time also causes Cargo to create a new file at the top
level: _Cargo.lock_. This file keeps track of the exact versions of
dependencies in your project. This project doesn't have dependencies, so the
file is a bit sparse. You won't ever need to change this file manually; Cargo
manages its contents for you.

We just built a project with `cargo build` and ran it with
`./target/debug/hello_cargo`, but we can also use `cargo run` to compile the
code and then run the resultant executable all in one command:

// ----------------------------------------

Using `cargo run` is more convenient than having to remember to run `cargo
```

```
build` and then use the whole path to the binary, so most developers use `cargo
run`.

Notice that this time we didn't see output indicating that Cargo was compiling
`hello_cargo`. Cargo figured out that the files hadn't changed, so it didn't
rebuild but just ran the binary. If you had modified your source code, Cargo
would have rebuilt the project before running it, and you would have seen this
output:
```

[rust_012] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Building for Release

Sumber: ht

Building for Release
When your project is finally ready for release, you can use `cargo build --release` to compile it with optimizations. This command will create an executable in _target/release_ instead of _target/debug_. The optimizations make your Rust code run faster, but turning them on lengthens the time it takes for your program to compile. This is why there are two different profiles: one for development, when you want to rebuild quickly and often, and another for building the final program you'll give to a user that won't be rebuilt repeatedly and that will run as fast as possible. If you're benchmarking your code's running time, be sure to run `cargo build --release` and benchmark with the executable in _target/release_.

[rust_013] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Leveraging Cargo's Conventions

Sumber: ht

Leveraging Cargo's Conventions
With simple projects, Cargo doesn't provide a lot of value over just using `rustc`, but it will prove its worth as your programs become more intricate. Once programs grow to multiple files or need a dependency, it's much easier to let Cargo coordinate the build.
Even though the `hello_cargo` project is simple, it now uses much of the real tooling you'll use in the rest of your Rust career. In fact, to work on any existing projects, you can use the following commands to check out the code using Git, change to that project's directory, and build:
For more information about Cargo, check out [its documentation][cargo].

[rust_014] Rust - Cargo Build System

Hello, Cargo! (Cargo.toml, cargo build, cargo run, cargo check): Summary

Sumber: ht

Summary
You're already off to a great start on your Rust journey! In this chapter, you learned how to:
- Install the latest stable version of Rust using `rustup`.
- Update to a newer Rust version.
- Open locally installed documentation.
- Write and run a "Hello, world!" program using `rustc` directly.
- Create and run a new project using the conventions of Cargo.
This is a great time to build a more substantial program to get used to reading

and writing Rust code. So, in Chapter 2, we'll build a guessing game program.
If you would rather start by learning how common programming concepts work in
Rust, see Chapter 3 and then return to Chapter 2.
[installation]: ch01-01-installation.html#installation
[toml]: https://toml.io
[appendix-e]: appendix-05-editions.html
[cargo]: https://doc.rust-lang.org/cargo/

[rust_015] Rust - Variables, Mutability & Types

Variables, Mutability, Shadowing & Constants: Variables and Mutability

Sumber: ht

Variables and Mutability
As mentioned in the ["Storing Values with
Variables"][storing-values-with-variables] section, by default,
variables are immutable. This is one of many nudges Rust gives you to write
your code in a way that takes advantage of the safety and easy concurrency that
Rust offers. However, you still have the option to make your variables mutable.
Let's explore how and why Rust encourages you to favor immutability and why
sometimes you might want to opt out.
When a variable is immutable, once a value is bound to a name, you can't change
that value. To illustrate this, generate a new project called _variables_ in
your _projects_ directory by using `cargo new variables`.
Then, in your new _variables_ directory, open _src/main.rs_ and replace its
code with the following code, which won't compile just yet:
Filename: src/main.rs
Save and run the program using `cargo run`. You should receive an error message
regarding an immutability error, as shown in this output:
This example shows how the compiler helps you find errors in your programs.
Compiler errors can be frustrating, but really they only mean your program
isn't safely doing what you want it to do yet; they do _not_ mean that you're
not a good programmer! Experienced Rustaceans still get compiler errors.
You received the error message `` cannot assign twice to immutable variable `x` `` because you tried to assign a second value to the immutable `x` variable.
It's important that we get compile-time errors when we attempt to change a
value that's designated as immutable, because this very situation can lead to
bugs. If one part of our code operates on the assumption that a value will
never change and another part of our code changes that value, it's possible
that the first part of the code won't do wh

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch03-common-programming-concepts/no-listing-01-variables-are-immutable/src/main.rs}}

// ----------------------------------------

This example shows how the compiler helps you find errors in your programs.
Compiler errors can be frustrating, but really they only mean your program
isn't safely doing what you want it to do yet; they do _not_ mean that you're
not a good programmer! Experienced Rustaceans still get compiler errors.

You received the error message `` cannot assign twice to immutable variable `x` `` because you tried to
assign a second value to the immutable `x` variable.

It's important that we get compile-time errors when we attempt to change a
```

```
value that's designated as immutable, because this very situation can lead to
bugs. If one part of our code operates on the assumption that a value will
never change and another part of our code changes that value, it's possible
that the first part of the code won't do what it was designed to do. The cause
of this kind of bug can be difficult to track down after the fact, especially
when the second piece of code changes the value only _sometimes_. The Rust
compiler guarantees that when you state that a value won't change, it really
won't change, so you don't have to keep track of it yourself. Your code is thus
easier to reason through.

But mutability can be very useful and can make code more convenient to write.
Although variables are immutable by default, you can make them mutable by
adding `mut` in front of the variable name as you did in [Chapter
2][storing-values-with-variables]<!-- ignore -->. Adding `mut` also conveys
intent to future readers of the code by indicating that other parts of the code
will be changing this variable's value.

For example, let's change _src/main.rs_ to the following:

<span class="filename">Filename: src/main.rs</span>

// ----------------------------------------

When we run the program now, we get this:
```

[rust_016] Rust - Variables, Mutability & Types

Variables, Mutability, Shadowing & Constants: Declaring Constants

Sumber: ht

Declaring Constants

Like immutable variables, _constants_ are values that are bound to a name and are not allowed to change, but there are a few differences between constants and variables.

First, you aren't allowed to use `mut` with constants. Constants aren't just immutable by default -- they're always immutable. You declare constants using the `const` keyword instead of the `let` keyword, and the type of the value _must_ be annotated. We'll cover types and type annotations in the next section, ["Data Types"][data-types], so don't worry about the details right now. Just know that you must always annotate the type.

Constants can be declared in any scope, including the global scope, which makes them useful for values that many parts of code need to know about.

The last difference is that constants may be set only to a constant expression, not the result of a value that could only be computed at runtime.

Here's an example of a constant declaration:

The constant's name is `THREE_HOURS_IN_SECONDS`, and its value is set to the result of multiplying 60 (the number of seconds in a minute) by 60 (the number of minutes in an hour) by 3 (the number of hours we want to count in this program). Rust's naming convention for constants is to use all uppercase with underscores between words. The compiler is able to evaluate a limited set of operations at compile time, which lets us choose to write out this value in a way that's easier to understand and verify, rather than setting this constant to the value 10,800. See the [Rust Reference's section on constant evaluation][const-eval] for more information on what operations can be used when declaring constants.

Constants are valid for the entire time a program runs, within the scope in which they were declared. This property makes const

Contoh Kode / Perintah CLI:

```
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;
```

[rust_017] Rust - Variables, Mutability & Types

Variables, Mutability, Shadowing & Constants: Shadowing

Sumber: ht

Shadowing
As you saw in the guessing game tutorial in [Chapter
2][comparing-the-guess-to-the-secret-number], you can declare a
new variable with the same name as a previous variable. Rustaceans say that the
first variable is _shadowed_ by the second, which means that the second
variable is what the compiler will see when you use the name of the variable.
In effect, the second variable overshadows the first, taking any uses of the
variable name to itself until either it itself is shadowed or the scope ends.
We can shadow a variable by using the same variable's name and repeating the
use of the `let` keyword as follows:
Filename: src/main.rs
This program first binds `x` to a value of `5`. Then, it creates a new variable
`x` by repeating `let x =`, taking the original value and adding `1` so that
the value of `x` is `6`. Then, within an inner scope created with the curly
brackets, the third `let` statement also shadows `x` and creates a new
variable, multiplying the previous value by `2` to give `x` a value of `12`.
When that scope is over, the inner shadowing ends and `x` returns to being `6`.
When we run this program, it will output the following:
Shadowing is different from marking a variable as `mut` because we'll get a
compile-time error if we accidentally try to reassign to this variable without
using the `let` keyword. By using `let`, we can perform a few transformations
on a value but have the variable be immutable after those transformations have
completed.
The other difference between `mut` and shadowing is that because we're
effectively creating a new variable when we use the `let` keyword again, we can
change the type of the value but reuse the same name. For example, say our
program asks a user to show how many spaces they want between some text by
inputti

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch03-common-programming-concepts/no-listing-03-shadowing/src/main.rs}}

// ----------------------------------------

Shadowing is different from marking a variable as `mut` because we'll get a
compile-time error if we accidentally try to reassign to this variable without
using the `let` keyword. By using `let`, we can perform a few transformations
on a value but have the variable be immutable after those transformations have
completed.

The other difference between `mut` and shadowing is that because we're
effectively creating a new variable when we use the `let` keyword again, we can
change the type of the value but reuse the same name. For example, say our
program asks a user to show how many spaces they want between some text by
inputting space characters, and then we want to store that input as a number:

// ----------------------------------------

The first `spaces` variable is a string type, and the second `spaces` variable
```

```
is a number type. Shadowing thus spares us from having to come up with
different names, such as `spaces_str` and `spaces_num`; instead, we can reuse
the simpler `spaces` name. However, if we try to use `mut` for this, as shown
here, we'll get a compile-time error:
```

[rust_018] Rust - Control Flow & Loops

Control Flow (if expressions, loop, while, for in ranges): Control Flow

Sumber: ht

Control Flow
The ability to run some code depending on whether a condition is `true` and the
ability to run some code repeatedly while a condition is `true` are basic
building blocks in most programming languages. The most common constructs that
let you control the flow of execution of Rust code are `if` expressions and
loops.

[rust_019] Rust - Control Flow & Loops

Control Flow (if expressions, loop, while, for in ranges): `if` Expressions

Sumber: ht

`if` Expressions
An `if` expression allows you to branch your code depending on conditions. You
provide a condition and then state, "If this condition is met, run this block
of code. If the condition is not met, do not run this block of code."
Create a new project called _branches_ in your _projects_ directory to explore
the `if` expression. In the _src/main.rs_ file, input the following:
Filename: src/main.rs
All `if` expressions start with the keyword `if`, followed by a condition. In
this case, the condition checks whether or not the variable `number` has a
value less than 5. We place the block of code to execute if the condition is
`true` immediately after the condition inside curly brackets. Blocks of code
associated with the conditions in `if` expressions are sometimes called _arms_,
just like the arms in `match` expressions that we discussed in the ["Comparing
the Guess to the Secret Number"][comparing-the-guess-to-the-secret-number] section of Chapter 2.
Optionally, we can also include an `else` expression, which we chose to do
here, to give the program an alternative block of code to execute should the
condition evaluate to `false`. If you don't provide an `else` expression and
the condition is `false`, the program will just skip the `if` block and move on
to the next bit of code.
Try running this code; you should see the following output:
Let's try changing the value of `number` to a value that makes the condition
`false` to see what happens:
Run the program again, and look at the output:
It's also worth noting that the condition in this code _must_ be a `bool`. If
the condition isn't a `bool`, we'll get an error. For example, try running the
following code:
Filename: src/main.rs
The `if` condition evaluates to a value of `3` this time, and Rust throws an

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch03-common-programming-concepts/no-listing-26-if-true/src/main.rs}}

// ----------------------------------------
```

```
Let's try changing the value of `number` to a value that makes the condition
`false` to see what happens:

// ----------------------------------------

Run the program again, and look at the output:
```

[rust_020] Rust - Control Flow & Loops

Control Flow (if expressions, loop, while, for in ranges): Repetition with Loops

Sumber: ht

Repetition with Loops
It's often useful to execute a block of code more than once. For this task,
Rust provides several _loops_, which will run through the code inside the loop
body to the end and then start immediately back at the beginning. To experiment
with loops, let's make a new project called _loops_.
Rust has three kinds of loops: `loop`, `while`, and `for`. Let's try each one.
Repeating Code with `loop`
The `loop` keyword tells Rust to execute a block of code over and over again
either forever or until you explicitly tell it to stop.
As an example, change the _src/main.rs_ file in your _loops_ directory to look
like this:
Filename: src/main.rs
When we run this program, we'll see `again!` printed over and over continuously
until we stop the program manually. Most terminals support the keyboard shortcut
ctrl-C to interrupt a program that is stuck in a continual
loop. Give it a try:
The symbol `^C` represents where you pressed ctrl-C.
You may or may not see the word `again!` printed after the `^C`, depending on
where the code was in the loop when it received the interrupt signal.
Fortunately, Rust also provides a way to break out of a loop using code. You
can place the `break` keyword within the loop to tell the program when to stop
executing the loop. Recall that we did this in the guessing game in the
["Quitting After a Correct Guess"][quitting-after-a-correct-guess] section of Chapter 2 to exit the program when the user won the game by
guessing the correct number.
We also used `continue` in the guessing game, which in a loop tells the program
to skip over any remaining code in this iteration of the loop and go to the
next iteration.
Returning Values from Loops
One of the uses of a `loop` is to retry an operation you know might fail, such
as checki

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch03-common-programming-concepts/no-listing-32-loop/src/main.rs}}

// ----------------------------------------

The symbol `^C` represents where you pressed <kbd>ctrl</kbd>-<kbd>C</kbd>.

You may or may not see the word `again!` printed after the `^C`, depending on
where the code was in the loop when it received the interrupt signal.

Fortunately, Rust also provides a way to break out of a loop using code. You
can place the `break` keyword within the loop to tell the program when to stop
```

```
executing the loop. Recall that we did this in the guessing game in the
["Quitting After a Correct Guess"][quitting-after-a-correct-guess]<!-- ignore
--> section of Chapter 2 to exit the program when the user won the game by
guessing the correct number.

We also used `continue` in the guessing game, which in a loop tells the program
to skip over any remaining code in this iteration of the loop and go to the
next iteration.

#### Returning Values from Loops

One of the uses of a `loop` is to retry an operation you know might fail, such
as checking whether a thread has completed its job. You might also need to pass
the result of that operation out of the loop to the rest of your code. To do
this, you can add the value you want returned after the `break` expression you
use to stop the loop; that value will be returned out of the loop so that you
can use it, as shown here:

// ----------------------------------------

Before the loop, we declare a variable named `counter` and initialize it to
`0`. Then, we declare a variable named `result` to hold the value returned from
the loop. On every iteration of the loop, we add `1` to the `counter` variable,
and then check whether the `counter` is equal to `10`. When it is, we use the
`break` keyword with the value `counter * 2`. After the loop, we use a
semicolon to end the statement that assigns the value to `result`. Finally, we
print the value in `result`, which in this case is `20`.

You can also `return` from inside a loop. While `break` only exits the current
loop, `return` always exits the current function.

<!-- Old headings. Do not remove or links may break. -->
<a id="loop-labels-to-disambiguate-between-multiple-loops"></a>

#### Disambiguating with Loop Labels

If you have loops within loops, `break` and `continue` apply to the innermost
loop at that point. You can optionally specify a _loop label_ on a loop that
you can then use with `break` or `continue` to specify that those keywords
apply to the labeled loop instead of the innermost loop. Loop labels must begin
with a single quote. Here's an example with two nested loops:
```

[rust_021] Rust - Control Flow & Loops

Control Flow (if expressions, loop, while, for in ranges): Summary

Sumber: ht

Summary
You made it! This was a sizable chapter: You learned about variables, scalar
and compound data types, functions, comments, `if` expressions, and loops! To
practice with the concepts discussed in this chapter, try building programs to
do the following:
- Convert temperatures between Fahrenheit and Celsius.
- Generate the *n*th Fibonacci number.
- Print the lyrics to the Christmas carol "The Twelve Days of Christmas,"
taking advantage of the repetition in the song.
When you're ready to move on, we'll talk about a concept in Rust that _doesn't_
commonly exist in other programming languages: ownership.
[comparing-the-guess-to-the-secret-number]:

ch02-00-guessing-game-tutorial.html#comparing-the-guess-to-the-secret-number
[quitting-after-a-correct-guess]: ch02-00-guessing-game-tutorial.html#quitting-after-a-correct-guess

[rust_022] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): What Is Ownership?

Sumber: ht

What Is Ownership?
Ownership is a set of rules that govern how a Rust program manages memory.
All programs have to manage the way they use a computer's memory while running.
Some languages have garbage collection that regularly looks for no-longer-used
memory as the program runs; in other languages, the programmer must explicitly
allocate and free the memory. Rust uses a third approach: Memory is managed
through a system of ownership with a set of rules that the compiler checks. If
any of the rules are violated, the program won't compile. None of the features
of ownership will slow down your program while it's running.
Because ownership is a new concept for many programmers, it does take some time
to get used to. The good news is that the more experienced you become with Rust
and the rules of the ownership system, the easier you'll find it to naturally
develop code that is safe and efficient. Keep at it!
When you understand ownership, you'll have a solid foundation for understanding
the features that make Rust unique. In this chapter, you'll learn ownership by
working through some examples that focus on a very common data structure:
strings.
> ### The Stack and the Heap
>
> Many programming languages don't require you to think about the stack and the
> heap very often. But in a systems programming language like Rust, whether a
> value is on the stack or the heap affects how the language behaves and why
> you have to make certain decisions. Parts of ownership will be described in
> relation to the stack and the heap later in this chapter, so here is a brief
> explanation in preparation.
>
> Both the stack and the heap are parts of memory available to your code to use
> at runtime, but they are structured in different ways. The stack stores
> values in the order it g

[rust_023] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): Ownership Rules

Sumber: ht

Ownership Rules
First, let's take a look at the ownership rules. Keep these rules in mind as we
work through the examples that illustrate them:
- Each value in Rust has an _owner_.
- There can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped.

[rust_024] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): Variable Scope

Sumber: ht

Variable Scope
Now that we're past basic Rust syntax, we won't include all the `fn main() {` code in the examples, so if you're following along, make sure to put the following examples inside a `main` function manually. As a result, our examples will be a bit more concise, letting us focus on the actual details rather than boilerplate code.
As a first example of ownership, we'll look at the scope of some variables. A _scope_ is the range within a program for which an item is valid. Take the following variable:
The variable `s` refers to a string literal, where the value of the string is hardcoded into the text of our program. The variable is valid from the point at which it's declared until the end of the current scope. Listing 4-1 shows a program with comments annotating where the variable `s` would be valid.
In other words, there are two important points in time here:
- When `s` comes _into_ scope, it is valid.
- It remains valid until it goes _out of_ scope.
At this point, the relationship between scopes and when variables are valid is similar to that in other programming languages. Now we'll build on top of this understanding by introducing the `String` type.

Contoh Kode / Perintah CLI:

```
let s = "hello";

// ------------------------------------------

{{#rustdoc_include ../listings/ch04-understanding-ownership/listing-04-01/src/main.rs:here}}
```

[rust_025] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): The `String` Type

Sumber: ht

The `String` Type
To illustrate the rules of ownership, we need a data type that is more complex than those we covered in the ["Data Types"][data-types] section of Chapter 3. The types covered previously are of a known size, can be stored on the stack and popped off the stack when their scope is over, and can be quickly and trivially copied to make a new, independent instance if another part of code needs to use the same value in a different scope. But we want to look at data that is stored on the heap and explore how Rust knows when to clean up that data, and the `String` type is a great example.
We'll concentrate on the parts of `String` that relate to ownership. These aspects also apply to other complex data types, whether they are provided by the standard library or created by you. We'll discuss non-ownership aspects of `String` in [Chapter 8][ch8].
We've already seen string literals, where a string value is hardcoded into our program. String literals are convenient, but they aren't suitable for every situation in which we may want to use text. One reason is that they're immutable. Another is that not every string value can be known when we write

our code: For example, what if we want to take user input and store it? It is
for these situations that Rust has the `String` type. This type manages
data allocated on the heap and as such is able to store an amount of text that
is unknown to us at compile time. You can create a `String` from a string
literal using the `from` function, like so:
The double colon `::` operator allows us to namespace this particular `from`
function under the `String` type rather than using some sort of name like
`string_from`. We'll discuss this syntax more in the ["Methods"][methods] section of Chapter 5, and when we talk about namespacin

Contoh Kode / Perintah CLI:

```
let s = String::from("hello");

// ----------------------------------------

{{#rustdoc_include
../listings/ch04-understanding-ownership/no-listing-01-can-mutate-string/src/main.rs:here}}
```

[rust_026] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): Memory and Allocation

Sumber: ht

Memory and Allocation
In the case of a string literal, we know the contents at compile time, so the
text is hardcoded directly into the final executable. This is why string
literals are fast and efficient. But these properties only come from the string
literal's immutability. Unfortunately, we can't put a blob of memory into the
binary for each piece of text whose size is unknown at compile time and whose
size might change while running the program.
With the `String` type, in order to support a mutable, growable piece of text,
we need to allocate an amount of memory on the heap, unknown at compile time,
to hold the contents. This means:
- The memory must be requested from the memory allocator at runtime.
- We need a way of returning this memory to the allocator when we're done with
our `String`.
That first part is done by us: When we call `String::from`, its implementation
requests the memory it needs. This is pretty much universal in programming
languages.
However, the second part is different. In languages with a _garbage collector
(GC)_, the GC keeps track of and cleans up memory that isn't being used
anymore, and we don't need to think about it. In most languages without a GC,
it's our responsibility to identify when memory is no longer being used and to
call code to explicitly free it, just as we did to request it. Doing this
correctly has historically been a difficult programming problem. If we forget,
we'll waste memory. If we do it too early, we'll have an invalid variable. If
we do it twice, that's a bug too. We need to pair exactly one `allocate` with
exactly one `free`.
Rust takes a different path: The memory is automatically returned once the
variable that owns it goes out of scope. Here's a version of our scope example
from Listing 4-1 using a `String`

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch04-understanding-ownership/no-listing-02-string-scope/src/main.rs:here}}
```

```
// ----------------------------------------

{{#rustdoc_include ../listings/ch04-understanding-ownership/listing-04-02/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch04-understanding-ownership/no-listing-03-string-move/src/main.rs:here}}
```

[rust_027] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): Ownership and Functions

Sumber: ht

Ownership and Functions
The mechanics of passing a value to a function are similar to those when assigning a value to a variable. Passing a variable to a function will move or copy, just as assignment does. Listing 4-3 has an example with some annotations showing where variables go into and out of scope.
If we tried to use `s` after the call to `takes_ownership`, Rust would throw a compile-time error. These static checks protect us from mistakes. Try adding code to `main` that uses `s` and `x` to see where you can use them and where the ownership rules prevent you from doing so.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch04-understanding-ownership/listing-04-03/src/main.rs}}
```

[rust_028] Rust - Ownership & Move Semantics

What is Ownership? (Stack vs Heap, Memory Allocation, Moves, Clones): Return Values and Scope

Sumber: ht

Return Values and Scope
Returning values can also transfer ownership. Listing 4-4 shows an example of a function that returns some value, with similar annotations as those in Listing 4-3.
The ownership of a variable follows the same pattern every time: Assigning a value to another variable moves it. When a variable that includes data on the heap goes out of scope, the value will be cleaned up by `drop` unless ownership of the data has been moved to another variable.
While this works, taking ownership and then returning ownership with every function is a bit tedious. What if we want to let a function use a value but not take ownership? It's quite annoying that anything we pass in also needs to be passed back if we want to use it again, in addition to any data resulting from the body of the function that we might want to return as well.
Rust does let us return multiple values using a tuple, as shown in Listing 4-5.
But this is too much ceremony and a lot of work for a concept that should be common. Luckily for us, Rust has a feature for using a value without transferring ownership: references.
[data-types]: ch03-02-data-types.html#data-types
[ch8]: ch08-02-strings.html
[traits]: ch10-02-traits.html
[derivable-traits]: appendix-03-derivable-traits.html
[methods]: ch05-03-method-syntax.html#methods
[paths-module-tree]: ch07-03-paths-for-referring-to-an-item-in-the-module-tree.html

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch04-understanding-ownership/listing-04-04/src/main.rs}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch04-understanding-ownership/listing-04-05/src/main.rs}}
```

[rust_029] Rust - References, Borrowing & Slices

References and Borrowing (Mutable References, Dangling References Rules): References and Borrowing

Sumber: ht

References and Borrowing
The issue with the tuple code in Listing 4-5 is that we have to return the
`String` to the calling function so that we can still use the `String` after
the call to `calculate_length`, because the `String` was moved into
`calculate_length`. Instead, we can provide a reference to the `String` value.
A reference is like a pointer in that it's an address we can follow to access
the data stored at that address; that data is owned by some other variable.
Unlike a pointer, a reference is guaranteed to point to a valid value of a
particular type for the life of that reference.
Here is how you would define and use a `calculate_length` function that has a
reference to an object as a parameter instead of taking ownership of the value:
First, notice that all the tuple code in the variable declaration and the
function return value is gone. Second, note that we pass `&s1` into
`calculate_length` and, in its definition, we take `&String` rather than
`String`. These ampersands represent references, and they allow you to refer to
some value without taking ownership of it. Figure 4-6 depicts this concept.
Figure 4-6: A diagram of `&String` `s` pointing at
`String` `s1`
> Note: The opposite of referencing by using `&` is _dereferencing_, which is
> accomplished with the dereference operator, `*`. We'll see some uses of the
> dereference operator in Chapter 8 and discuss details of dereferencing in
> Chapter 15.
Let's take a closer look at the function call here:
The `&s1` syntax lets us create a reference that _refers_ to the value of `s1`
but does not own it. Because the reference does not own it, the value it points
to will not be dropped when the reference stops being used.
Likewise, the signature of the function uses `&` to indicate that the type of
the par

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch04-understanding-ownership/no-listing-07-reference/src/main.rs:all}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch04-understanding-ownership/no-listing-07-reference/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch04-understanding-ownership/no-listing-08-reference-with-annotations/src/main.rs:here}}
```

[rust_030] Rust - References, Borrowing & Slices

References and Borrowing (Mutable References, Dangling References Rules): Mutable References

Sumber: ht

Mutable References
We can fix the code from Listing 4-6 to allow us to modify a borrowed value with just a few small tweaks that use, instead, a _mutable reference_:
First, we change `s` to be `mut`. Then, we create a mutable reference with `&mut s` where we call the `change` function and update the function signature to accept a mutable reference with `some_string: &mut String`. This makes it very clear that the `change` function will mutate the value it borrows.
Mutable references have one big restriction: If you have a mutable reference to a value, you can have no other references to that value. This code that attempts to create two mutable references to `s` will fail:
Here's the error:
This error says that this code is invalid because we cannot borrow `s` as mutable more than once at a time. The first mutable borrow is in `r1` and must last until it's used in the `println!`, but between the creation of that mutable reference and its usage, we tried to create another mutable reference in `r2` that borrows the same data as `r1`.
The restriction preventing multiple mutable references to the same data at the same time allows for mutation but in a very controlled fashion. It's something that new Rustaceans struggle with because most languages let you mutate whenever you'd like. The benefit of having this restriction is that Rust can prevent data races at compile time. A _data race_ is similar to a race condition and happens when these three behaviors occur:
- Two or more pointers access the same data at the same time.
- At least one of the pointers is being used to write to the data.
- There's no mechanism being used to synchronize access to the data.
Data races cause undefined behavior and can be difficult to diagnose and fix when you're trying to track them down at

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch04-understanding-ownership/no-listing-09-fixes-listing-04-06/src/main.rs}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch04-understanding-ownership/no-listing-10-multiple-mut-not-allowed/src/main.rs:here}}

// ----------------------------------------

This error says that this code is invalid because we cannot borrow `s` as
mutable more than once at a time. The first mutable borrow is in `r1` and must
last until it's used in the `println!`, but between the creation of that
mutable reference and its usage, we tried to create another mutable reference
in `r2` that borrows the same data as `r1`.

The restriction preventing multiple mutable references to the same data at the
same time allows for mutation but in a very controlled fashion. It's something
that new Rustaceans struggle with because most languages let you mutate
whenever you'd like. The benefit of having this restriction is that Rust can
prevent data races at compile time. A _data race_ is similar to a race
condition and happens when these three behaviors occur:
```

```
- Two or more pointers access the same data at the same time.
- At least one of the pointers is being used to write to the data.
- There's no mechanism being used to synchronize access to the data.

Data races cause undefined behavior and can be difficult to diagnose and fix
when you're trying to track them down at runtime; Rust prevents this problem by
refusing to compile code with data races!

As always, we can use curly brackets to create a new scope, allowing for
multiple mutable references, just not _simultaneous_ ones:
```

[rust_031] Rust - References, Borrowing & Slices

References and Borrowing (Mutable References, Dangling References Rules): Dangling References

Sumber: ht

Dangling References

In languages with pointers, it's easy to erroneously create a _dangling pointer_ -- a pointer that references a location in memory that may have been given to someone else -- by freeing some memory while preserving a pointer to that memory. In Rust, by contrast, the compiler guarantees that references will never be dangling references: If you have a reference to some data, the compiler will ensure that the data will not go out of scope before the reference to the data does.

Let's try to create a dangling reference to see how Rust prevents them with a compile-time error:

Here's the error:

This error message refers to a feature we haven't covered yet: lifetimes. We'll discuss lifetimes in detail in Chapter 10. But, if you disregard the parts about lifetimes, the message does contain the key to why this code is a problem:

Let's take a closer look at exactly what's happening at each stage of our `dangle` code:

Because `s` is created inside `dangle`, when the code of `dangle` is finished, `s` will be deallocated. But we tried to return a reference to it. That means this reference would be pointing to an invalid `String`. That's no good! Rust won't let us do this.

The solution here is to return the `String` directly:

This works without any problems. Ownership is moved out, and nothing is deallocated.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch04-understanding-ownership/no-listing-14-dangling-reference/src/main.rs}}

// ----------------------------------------

This error message refers to a feature we haven't covered yet: lifetimes. We'll
discuss lifetimes in detail in Chapter 10. But, if you disregard the parts
about lifetimes, the message does contain the key to why this code is a problem:

// ----------------------------------------

Let's take a closer look at exactly what's happening at each stage of our
`dangle` code:

<Listing file-name="src/main.rs">
```

[rust_032] Rust - References, Borrowing & Slices

References and Borrowing (Mutable References, Dangling References Rules): The Rules of References

Sumber: ht

The Rules of References
Let's recap what we've discussed about references:
- At any given time, you can have _either_ one mutable reference _or_ any number of immutable references.
- References must always be valid.
Next, we'll look at a different kind of reference: slices.

[rust_033] Rust - Structs & Method Syntax

Defining and Instantiating Structs (Tuple Structs, impl blocks): Defining and Instantiating Structs

Sumber: ht

Defining and Instantiating Structs
Structs are similar to tuples, discussed in ["The Tuple Type"][tuples] section, in that both hold multiple related values. Like tuples, the
pieces of a struct can be different types. Unlike with tuples, in a struct you'll name each piece of data so it's clear what the values mean. Adding these names means that structs are more flexible than tuples: You don't have to rely on the order of the data to specify or access the values of an instance.
To define a struct, we enter the keyword `struct` and name the entire struct. A struct's name should describe the significance of the pieces of data being grouped together. Then, inside curly brackets, we define the names and types of the pieces of data, which we call _fields_. For example, Listing 5-1 shows a struct that stores information about a user account.
To use a struct after we've defined it, we create an _instance_ of that struct by specifying concrete values for each of the fields. We create an instance by stating the name of the struct and then add curly brackets containing _`key: value`_ pairs, where the keys are the names of the fields and the values are the data we want to store in those fields. We don't have to specify the fields in the same order in which we declared them in the struct. In other words, the struct definition is like a general template for the type, and instances fill in that template with particular data to create values of the type. For example, we can declare a particular user as shown in Listing 5-2.
To get a specific value from a struct, we use dot notation. For example, to access this user's email address, we use `user1.email`. If the instance is mutable, we can change a value by using the dot notation and assigning into a particular field. Listing 5-3 show

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/listing-05-01/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/listing-05-02/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
```

```
../listings/ch05-using-structs-to-structure-related-data/listing-05-03/src/main.rs:here}}
```

[rust_034] Rust - Structs & Method Syntax

Defining and Instantiating Structs (Tuple Structs, impl blocks): Using the Field Init Shorthand

Sumber: ht

Using the Field Init Shorthand
Because the parameter names and the struct field names are exactly the same in Listing 5-4, we can use the _field init shorthand_ syntax to rewrite `build_user` so that it behaves exactly the same but doesn't have the repetition of `username` and `email`, as shown in Listing 5-5.
Here, we're creating a new instance of the `User` struct, which has a field named `email`. We want to set the `email` field's value to the value in the `email` parameter of the `build_user` function. Because the `email` field and the `email` parameter have the same name, we only need to write `email` rather than `email: email`.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/listing-05-05/src/main.rs:here}}
```

[rust_035] Rust - Structs & Method Syntax

Defining and Instantiating Structs (Tuple Structs, impl blocks): Creating Instances with Struct Update Syntax

Sumber: ht

Creating Instances with Struct Update Syntax
It's often useful to create a new instance of a struct that includes most of the values from another instance of the same type, but changes some of them. You can do this using struct update syntax.
First, in Listing 5-6 we show how to create a new `User` instance in `user2` in the regular way, without the update syntax. We set a new value for `email` but otherwise use the same values from `user1` that we created in Listing 5-2.
Using struct update syntax, we can achieve the same effect with less code, as shown in Listing 5-7. The syntax `..` specifies that the remaining fields not explicitly set should have the same value as the fields in the given instance.
The code in Listing 5-7 also creates an instance in `user2` that has a different value for `email` but has the same values for the `username`, `active`, and `sign_in_count` fields from `user1`. The `..user1` must come last to specify that any remaining fields should get their values from the corresponding fields in `user1`, but we can choose to specify values for as many fields as we want in any order, regardless of the order of the fields in the struct's definition.
Note that the struct update syntax uses `=` like an assignment; this is because it moves the data, just as we saw in the ["Variables and Data Interacting with Move"][move] section. In this example, we can no longer use `user1` after creating `user2` because the `String` in the `username` field of `user1` was moved into `user2`. If we had given `user2` new `String` values for both `email` and `username`, and thus only used the `active` and `sign_in_count` values from `user1`, then `user1` would still be valid after creating `user2`.
Both `active` and `sign_in_count` are types that implement the `Copy` trai

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/listing-05-06/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/listing-05-07/src/main.rs:here}}
```

[rust_036] Rust - Structs & Method Syntax

Defining and Instantiating Structs (Tuple Structs, impl blocks): Creating Different Types with Tuple Structs

Sumber: ht

Creating Different Types with Tuple Structs
Rust also supports structs that look similar to tuples, called _tuple structs_. Tuple structs have the added meaning the struct name provides but don't have names associated with their fields; rather, they just have the types of the fields. Tuple structs are useful when you want to give the whole tuple a name and make the tuple a different type from other tuples, and when naming each field as in a regular struct would be verbose or redundant.
To define a tuple struct, start with the `struct` keyword and the struct name followed by the types in the tuple. For example, here we define and use two tuple structs named `Color` and `Point`:
Note that the `black` and `origin` values are different types because they're instances of different tuple structs. Each struct you define is its own type, even though the fields within the struct might have the same types. For example, a function that takes a parameter of type `Color` cannot take a `Point` as an argument, even though both types are made up of three `i32` values. Otherwise, tuple struct instances are similar to tuples in that you can destructure them into their individual pieces, and you can use a `.` followed by the index to access an individual value. Unlike tuples, tuple structs require you to name the type of the struct when you destructure them. For example, we would write `let Point(x, y, z) = origin;` to destructure the values in the `origin` point into variables named `x`, `y`, and `z`.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/no-listing-01-tuple-structs/src/main.rs}}
```

[rust_037] Rust - Structs & Method Syntax

Defining and Instantiating Structs (Tuple Structs, impl blocks): Defining Unit-Like Structs

Sumber: ht

Defining Unit-Like Structs
You can also define structs that don't have any fields! These are called _unit-like structs_ because they behave similarly to `()`, the unit type that we mentioned in ["The Tuple Type"][tuples] section. Unit-like structs can be useful when you need to implement a trait on some type but don't have any data that you want to store in the type itself. We'll discuss traits in Chapter 10. Here's an example of declaring and instantiating a unit struct named `AlwaysEqual`:
To define `AlwaysEqual`, we use the `struct` keyword, the name we want, and then a semicolon. No need for curly brackets or parentheses! Then, we can get

an instance of `AlwaysEqual` in the `subject` variable in a similar way: using the name we defined, without any curly brackets or parentheses. Imagine that later we'll implement behavior for this type such that every instance of `AlwaysEqual` is always equal to every instance of any other type, perhaps to have a known result for testing purposes. We wouldn't need any data to implement that behavior! You'll see in Chapter 10 how to define traits and implement them on any type, including unit-like structs.
> ### Ownership of Struct Data
>
> In the `User` struct definition in Listing 5-1, we used the owned `String`
> type rather than the `&str` string slice type. This is a deliberate choice
> because we want each instance of this struct to own all of its data and for
> that data to be valid for as long as the entire struct is valid.
>
> It's also possible for structs to store references to data owned by something
> else, but to do so requires the use of _lifetimes_, a Rust feature that we'll
> discuss in Chapter 10. Lifetimes ensure that the data referenced by a struct
> is valid for as long as the struct is. Let's say you try to

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch05-using-structs-to-structure-related-data/no-listing-04-unit-like-structs/src/main.rs}}

// ----------------------------------------

> struct User {
>     active: bool,
>     username: &str,
>     email: &str,
>     sign_in_count: u64,
> }
>
> fn main() {
>     let user1 = User {
>         active: true,
>         username: "someusername123",
>         email: "someone@example.com",
>         sign_in_count: 1,
>     };
> }
>
```

[rust_038] Rust - Enums & Pattern Matching

The match Control Flow Construct (Option<T>, Exhaustive Matching, if-let, let-else): The `match` Control Flow Construct

Sumber: ht

The `match` Control Flow Construct
Rust has an extremely powerful control flow construct called `match` that allows you to compare a value against a series of patterns and then execute code based on which pattern matches. Patterns can be made up of literal values, variable names, wildcards, and many other things; [Chapter 19][ch19-00-patterns] covers all the different kinds of patterns and what they do. The power of `match` comes from the expressiveness of the patterns and the fact that the compiler confirms that all possible cases are handled.

Think of a `match` expression as being like a coin-sorting machine: Coins slide down a track with variously sized holes along it, and each coin falls through the first hole it encounters that it fits into. In the same way, values go through each pattern in a `match`, and at the first pattern the value "fits," the value falls into the associated code block to be used during execution.
Speaking of coins, let's use them as an example using `match`! We can write a function that takes an unknown US coin and, in a similar way as the counting machine, determines which coin it is and returns its value in cents, as shown in Listing 6-3.
Let's break down the `match` in the `value_in_cents` function. First, we list the `match` keyword followed by an expression, which in this case is the value `coin`. This seems very similar to a conditional expression used with `if`, but there's a big difference: With `if`, the condition needs to evaluate to a Boolean value, but here it can be any type. The type of `coin` in this example is the `Coin` enum that we defined on the first line.
Next are the `match` arms. An arm has two parts: a pattern and some code. The first arm here has a pattern that is the value `Coin::Penny` and then the `=>` operator

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch06-enums-and-pattern-matching/listing-06-03/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-08-match-arm-multiple-lines/src/main.rs:here}}
```

[rust_039] Rust - Enums & Pattern Matching

The match Control Flow Construct (Option<T>, Exhaustive Matching, if-let, let-else): Patterns That Bind to Values

Sumber: ht

Patterns That Bind to Values
Another useful feature of match arms is that they can bind to the parts of the values that match the pattern. This is how we can extract values out of enum variants.
As an example, let's change one of our enum variants to hold data inside it. From 1999 through 2008, the United States minted quarters with different designs for each of the 50 states on one side. No other coins got state designs, so only quarters have this extra value. We can add this information to our `enum` by changing the `Quarter` variant to include a `UsState` value stored inside it, which we've done in Listing 6-4.
Let's imagine that a friend is trying to collect all 50 state quarters. While we sort our loose change by coin type, we'll also call out the name of the state associated with each quarter so that if it's one our friend doesn't have, they can add it to their collection.
In the match expression for this code, we add a variable called `state` to the pattern that matches values of the variant `Coin::Quarter`. When a `Coin::Quarter` matches, the `state` variable will bind to the value of that quarter's state. Then, we can use `state` in the code for that arm, like so:
If we were to call `value_in_cents(Coin::Quarter(UsState::Alaska))`, `coin` would be `Coin::Quarter(UsState::Alaska)`. When we compare that value with each of the match arms, none of them match until we reach `Coin::Quarter(state)`. At

that point, the binding for `state` will be the value `UsState::Alaska`. We can then use that binding in the `println!` expression, thus getting the inner state value out of the `Coin` enum variant for `Quarter`.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch06-enums-and-pattern-matching/listing-06-04/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-09-variable-in-pattern/src/main.rs:here}}
```

[rust_040] Rust - Enums & Pattern Matching

The match Control Flow Construct (Option<T>, Exhaustive Matching, if-let, let-else): The `Option<T>` `match` Pattern

Sumber: ht

The `Option` `match` Pattern

In the previous section, we wanted to get the inner `T` value out of the `Some` case when using `Option`; we can also handle `Option` using `match`, as we did with the `Coin` enum! Instead of comparing coins, we'll compare the variants of `Option`, but the way the `match` expression works remains the same.

Let's say we want to write a function that takes an `Option` and, if there's a value inside, adds 1 to that value. If there isn't a value inside, the function should return the `None` value and not attempt to perform any operations.

This function is very easy to write, thanks to `match`, and will look like Listing 6-5.

`">

Let's examine the first execution of `plus_one` in more detail. When we call `plus_one(five)`, the variable `x` in the body of `plus_one` will have the value `Some(5)`. We then compare that against each match arm:

The `Some(5)` value doesn't match the pattern `None`, so we continue to the next arm:

Does `Some(5)` match `Some(i)`? It does! We have the same variant. The `i` binds to the value contained in `Some`, so `i` takes the value `5`. The code in the match arm is then executed, so we add 1 to the value of `i` and create a new `Some` value with our total `6` inside.

Now let's consider the second call of `plus_one` in Listing 6-5, where `x` is `None`. We enter the `match` and compare to the first arm:

It matches! There's no value to add to, so the program stops and returns the `None` value on the right side of `=>`. Because the first arm matched, no other arms are compared.

Combining `match` and enums is useful in many situations. You'll see this pattern a lot in Rust code: `match` against an enum, bind a variable to the data inside, and then execute code based on it. It's a bit tricky at first, but once you get us

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch06-enums-and-pattern-matching/listing-06-05/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch06-enums-and-pattern-matching/listing-06-05/src/main.rs:first_arm}}
```

```
// ----------------------------------------

{{#rustdoc_include ../listings/ch06-enums-and-pattern-matching/listing-06-05/src/main.rs:second_arm}}
```

[rust_041] Rust - Enums & Pattern Matching

The match Control Flow Construct (Option<T>, Exhaustive Matching, if-let, let-else): Matches Are Exhaustive

Sumber: ht

Matches Are Exhaustive

There's one other aspect of `match` we need to discuss: The arms' patterns must cover all possibilities. Consider this version of our `plus_one` function, which has a bug and won't compile:

We didn't handle the `None` case, so this code will cause a bug. Luckily, it's a bug Rust knows how to catch. If we try to compile this code, we'll get this error:

Rust knows that we didn't cover every possible case and even knows which pattern we forgot! Matches in Rust are _exhaustive_: We must exhaust every last possibility in order for the code to be valid. Especially in the case of `Option`, when Rust prevents us from forgetting to explicitly handle the `None` case, it protects us from assuming that we have a value when we might have null, thus making the billion-dollar mistake discussed earlier impossible.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-10-non-exhaustive-match/src/main.rs:here}}
```

[rust_042] Rust - Enums & Pattern Matching

The match Control Flow Construct (Option<T>, Exhaustive Matching, if-let, let-else): Catch-All Patterns and the `_` Placeholder

Sumber: ht

Catch-All Patterns and the `_` Placeholder

Using enums, we can also take special actions for a few particular values, but for all other values take one default action. Imagine we're implementing a game where, if you roll a 3 on a dice roll, your player doesn't move but instead gets a fancy new hat. If you roll a 7, your player loses a fancy hat. For all other values, your player moves that number of spaces on the game board. Here's a `match` that implements that logic, with the result of the dice roll hardcoded rather than a random value, and all other logic represented by functions without bodies because actually implementing them is out of scope for this example:

For the first two arms, the patterns are the literal values `3` and `7`. For the last arm that covers every other possible value, the pattern is the variable we've chosen to name `other`. The code that runs for the `other` arm uses the variable by passing it to the `move_player` function.

This code compiles, even though we haven't listed all the possible values a `u8` can have, because the last pattern will match all values not specifically listed. This catch-all pattern meets the requirement that `match` must be exhaustive. Note that we have to put the catch-all arm last because the patterns are evaluated in order. If we had put the catch-all arm earlier, the other arms would never run, so Rust will warn us if we add arms after a

catch-all!
Rust also has a pattern we can use when we want a catch-all but don't want to
use the value in the catch-all pattern: `_` is a special pattern that matches
any value and does not bind to that value. This tells Rust we aren't going to
use the value, so Rust won't warn us about an unused variable.
Let's change the rules of the game: Now, if you roll anything other tha

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-15-binding-catchall/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-16-underscore-catchall/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch06-enums-and-pattern-matching/no-listing-17-underscore-unit/src/main.rs:here}}
```

[rust_043] Rust - Error Handling (Result & ? Operator)

Recoverable Errors with Result<T, E> and Propagating Errors with ?: Recoverable Errors with `Result`

Sumber: ht

Recoverable Errors with `Result`
Most errors aren't serious enough to require the program to stop entirely.
Sometimes when a function fails, it's for a reason that you can easily interpret
and respond to. For example, if you try to open a file and that operation fails
because the file doesn't exist, you might want to create the file instead of
terminating the process.
Recall from ["Handling Potential Failure with `Result`"][handle_failure] in Chapter 2 that the `Result` enum is defined as having two
variants, `Ok` and `Err`, as follows:
The `T` and `E` are generic type parameters: We'll discuss generics in more
detail in Chapter 10. What you need to know right now is that `T` represents
the type of the value that will be returned in a success case within the `Ok`
variant, and `E` represents the type of the error that will be returned in a
failure case within the `Err` variant. Because `Result` has these generic type
parameters, we can use the `Result` type and the functions defined on it in
many different situations where the success value and error value we want to
return may differ.
Let's call a function that returns a `Result` value because the function could
fail. In Listing 9-3, we try to open a file.
The return type of `File::open` is a `Result`. The generic parameter `T`
has been filled in by the implementation of `File::open` with the type of the
success value, `std::fs::File`, which is a file handle. The type of `E` used in
the error value is `std::io::Error`. This return type means the call to
`File::open` might succeed and return a file handle that we can read from or
write to. The function call also might fail: For example, the file might not
exist, or we might not have permission to access the file. The `File::open`
function needs to have a way to tell u

Contoh Kode / Perintah CLI:

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}

// ----------------------------------------

{{#rustdoc_include ../listings/ch09-error-handling/listing-09-03/src/main.rs}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch09-error-handling/listing-09-04/src/main.rs}}
```

[rust_044] Rust - Error Handling (Result & ? Operator)

Recoverable Errors with Result<T, E> and Propagating Errors with ?: Matching on Different Errors

Sumber: ht

Matching on Different Errors
The code in Listing 9-4 will `panic!` no matter why `File::open` failed.
However, we want to take different actions for different failure reasons. If
`File::open` failed because the file doesn't exist, we want to create the file
and return the handle to the new file. If `File::open` failed for any other
reason -- for example, because we didn't have permission to open the file -- we still
want the code to `panic!` in the same way it did in Listing 9-4. For this, we
add an inner `match` expression, shown in Listing 9-5.
The type of the value that `File::open` returns inside the `Err` variant is
`io::Error`, which is a struct provided by the standard library. This struct
has a method, `kind`, that we can call to get an `io::ErrorKind` value. The
enum `io::ErrorKind` is provided by the standard library and has variants
representing the different kinds of errors that might result from an `io`
operation. The variant we want to use is `ErrorKind::NotFound`, which indicates
the file we're trying to open doesn't exist yet. So, we match on
`greeting_file_result`, but we also have an inner match on `error.kind()`.
The condition we want to check in the inner match is whether the value returned
by `error.kind()` is the `NotFound` variant of the `ErrorKind` enum. If it is,
we try to create the file with `File::create`. However, because `File::create`
could also fail, we need a second arm in the inner `match` expression. When the
file can't be created, a different error message is printed. The second arm of
the outer `match` stays the same, so the program panics on any error besides
the missing file error.
> #### Alternatives to Using `match` with `Result`
>
> That's a lot of `match`! The `match` expression is very useful but also very
> much a primitive. In

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch09-error-handling/listing-09-05/src/main.rs}}

// ----------------------------------------

> use std::fs::File;
> use std::io::ErrorKind;
>
> fn main() {
>     let greeting_file = File::open("hello.txt").unwrap_or_else(|error| {
```

```
>         if error.kind() == ErrorKind::NotFound {
>             File::create("hello.txt").unwrap_or_else(|error| {
>                 panic!("Problem creating the file: {error:?}");
>             })
>         } else {
>             panic!("Problem opening the file: {error:?}");
>         }
>     });
> }
>

// ----------------------------------------

{{#rustdoc_include ../listings/ch09-error-handling/no-listing-04-unwrap/src/main.rs}}
```

[rust_045] Rust - Error Handling (Result & ? Operator)

Recoverable Errors with Result<T, E> and Propagating Errors with ?: Propagating Errors

Sumber: ht

Propagating Errors

When a function's implementation calls something that might fail, instead of handling the error within the function itself, you can return the error to the calling code so that it can decide what to do. This is known as _propagating_ the error and gives more control to the calling code, where there might be more information or logic that dictates how the error should be handled than what you have available in the context of your code.

For example, Listing 9-6 shows a function that reads a username from a file. If the file doesn't exist or can't be read, this function will return those errors to the code that called the function.

This function can be written in a much shorter way, but we're going to start by doing a lot of it manually in order to explore error handling; at the end, we'll show the shorter way. Let's look at the return type of the function first: `Result`. This means the function is returning a value of the type `Result`, where the generic parameter `T` has been filled in with the concrete type `String` and the generic type `E` has been filled in with the concrete type `io::Error`.

If this function succeeds without any problems, the code that calls this function will receive an `Ok` value that holds a `String` -- the `username` that this function read from the file. If this function encounters any problems, the calling code will receive an `Err` value that holds an instance of `io::Error` that contains more information about what the problems were. We chose `io::Error` as the return type of this function because that happens to be the type of the error value returned from both of the operations we're calling in this function's body that might fail: the `File::open` function and the `read_to_string` method.

The body of the function start

Contoh Kode / Perintah CLI:

```
{{#include ../listings/ch09-error-handling/listing-09-06/src/main.rs:here}}

// ----------------------------------------

{{#include ../listings/ch09-error-handling/listing-09-07/src/main.rs:here}}

// ----------------------------------------
```

```
{{#include ../listings/ch09-error-handling/listing-09-08/src/main.rs:here}}
```

[rust_046] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Defining Shared Behavior with Traits

Sumber: ht

Defining Shared Behavior with Traits
A _trait_ defines the functionality a particular type has and can share with other types. We can use traits to define shared behavior in an abstract way. We can use _trait bounds_ to specify that a generic type can be any type that has certain behavior.
> Note: Traits are similar to a feature often called _interfaces_ in other > languages, although with some differences.

[rust_047] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Defining a Trait

Sumber: ht

Defining a Trait
A type's behavior consists of the methods we can call on that type. Different types share the same behavior if we can call the same methods on all of those types. Trait definitions are a way to group method signatures together to define a set of behaviors necessary to accomplish some purpose.
For example, let's say we have multiple structs that hold various kinds and amounts of text: a `NewsArticle` struct that holds a news story filed in a particular location and a `SocialPost` that can have, at most, 280 characters along with metadata that indicates whether it was a new post, a repost, or a reply to another post.
We want to make a media aggregator library crate named `aggregator` that can display summaries of data that might be stored in a `NewsArticle` or `SocialPost` instance. To do this, we need a summary from each type, and we'll request that summary by calling a `summarize` method on an instance. Listing 10-12 shows the definition of a public `Summary` trait that expresses this behavior.
Here, we declare a trait using the `trait` keyword and then the trait's name, which is `Summary` in this case. We also declare the trait as `pub` so that crates depending on this crate can make use of this trait too, as we'll see in a few examples. Inside the curly brackets, we declare the method signatures that describe the behaviors of the types that implement this trait, which in this case is `fn summarize(&self) -> String`.
After the method signature, instead of providing an implementation within curly brackets, we use a semicolon. Each type implementing this trait must provide its own custom behavior for the body of the method. The compiler will enforce that any type that has the `Summary` trait will have the method `summarize` defined with this signatur

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-12/src/lib.rs}}
```

[rust_048] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Implementing a Trait on a Type

Sumber: ht

Implementing a Trait on a Type

Now that we've defined the desired signatures of the `Summary` trait's methods, we can implement it on the types in our media aggregator. Listing 10-13 shows an implementation of the `Summary` trait on the `NewsArticle` struct that uses the headline, the author, and the location to create the return value of `summarize`. For the `SocialPost` struct, we define `summarize` as the username followed by the entire text of the post, assuming that the post content is already limited to 280 characters.

Implementing a trait on a type is similar to implementing regular methods. The difference is that after `impl`, we put the trait name we want to implement, then use the `for` keyword, and then specify the name of the type we want to implement the trait for. Within the `impl` block, we put the method signatures that the trait definition has defined. Instead of adding a semicolon after each signature, we use curly brackets and fill in the method body with the specific behavior that we want the methods of the trait to have for the particular type.

Now that the library has implemented the `Summary` trait on `NewsArticle` and `SocialPost`, users of the crate can call the trait methods on instances of `NewsArticle` and `SocialPost` in the same way we call regular methods. The only difference is that the user must bring the trait into scope as well as the types. Here's an example of how a binary crate could use our `aggregator` library crate:

This code prints `1 new post: horse_ebooks: of course, as you probably already know, people`.

Other crates that depend on the `aggregator` crate can also bring the `Summary` trait into scope to implement `Summary` on their own types. One restriction to note is that we can implement a trait on a type only if either

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-13/src/lib.rs:here}}

// ------------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-01-calling-trait-method/src/main.rs}}
```

[rust_049] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Using Default Implementations

Sumber: ht

Using Default Implementations

Sometimes it's useful to have default behavior for some or all of the methods in a trait instead of requiring implementations for all methods on every type. Then, as we implement the trait on a particular type, we can keep or override each method's default behavior.

In Listing 10-14, we specify a default string for the `summarize` method of the `Summary` trait instead of only defining the method signature, as we did in Listing 10-12.

To use a default implementation to summarize instances of `NewsArticle`, we

specify an empty `impl` block with `impl Summary for NewsArticle {}`.
Even though we're no longer defining the `summarize` method on `NewsArticle` directly, we've provided a default implementation and specified that `NewsArticle` implements the `Summary` trait. As a result, we can still call the `summarize` method on an instance of `NewsArticle`, like this:
This code prints `New article available! (Read more...)`.
Creating a default implementation doesn't require us to change anything about the implementation of `Summary` on `SocialPost` in Listing 10-13. The reason is that the syntax for overriding a default implementation is the same as the syntax for implementing a trait method that doesn't have a default implementation.
Default implementations can call other methods in the same trait, even if those other methods don't have a default implementation. In this way, a trait can provide a lot of useful functionality and only require implementors to specify a small part of it. For example, we could define the `Summary` trait to have a `summarize_author` method whose implementation is required, and then define a `summarize` method that has a default implementation that calls the `summarize_author` method:
To use this version of `Summary

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-14/src/lib.rs:here}}

// ---------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-02-calling-default-impl/src/main.rs:here}}

// ---------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-03-default-impl-calls-other-methods/src/li
b.rs:here}}
```

[rust_050] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Using Traits as Parameters

Sumber: ht

Using Traits as Parameters
Now that you know how to define and implement traits, we can explore how to use traits to define functions that accept many different types. We'll use the `Summary` trait we implemented on the `NewsArticle` and `SocialPost` types in Listing 10-13 to define a `notify` function that calls the `summarize` method on its `item` parameter, which is of some type that implements the `Summary` trait. To do this, we use the `impl Trait` syntax, like this:
Instead of a concrete type for the `item` parameter, we specify the `impl` keyword and the trait name. This parameter accepts any type that implements the specified trait. In the body of `notify`, we can call any methods on `item` that come from the `Summary` trait, such as `summarize`. We can call `notify` and pass in any instance of `NewsArticle` or `SocialPost`. Code that calls the function with any other type, such as a `String` or an `i32`, won't compile, because those types don't implement `Summary`.
Trait Bound Syntax

The `impl Trait` syntax works for straightforward cases but is actually syntax sugar for a longer form known as a _trait bound_; it looks like this:
This longer form is equivalent to the example in the previous section but is more verbose. We place trait bounds with the declaration of the generic type parameter after a colon and inside angle brackets.
The `impl Trait` syntax is convenient and makes for more concise code in simple cases, while the fuller trait bound syntax can express more complexity in other cases. For example, we can have two parameters that implement `Summary`. Doing so with the `impl Trait` syntax looks like this:
Using `impl Trait` is appropriate if we want this function to allow `item1` and `item2` to have different types (as long as both types implem

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-04-traits-as-parameters/src/lib.rs:here}}

// ---------------------------------------

pub fn notify<T: Summary>(item: &T) {
    println!("Breaking news! {}", item.summarize());
}

// ---------------------------------------

pub fn notify(item1: &impl Summary, item2: &impl Summary) {
```

[rust_051] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Returning Types That Implement Traits

Sumber: ht

Returning Types That Implement Traits
We can also use the `impl Trait` syntax in the return position to return a value of some type that implements a trait, as shown here:
By using `impl Summary` for the return type, we specify that the `returns_summarizable` function returns some type that implements the `Summary` trait without naming the concrete type. In this case, `returns_summarizable` returns a `SocialPost`, but the code calling this function doesn't need to know that.
The ability to specify a return type only by the trait it implements is especially useful in the context of closures and iterators, which we cover in Chapter 13. Closures and iterators create types that only the compiler knows or types that are very long to specify. The `impl Trait` syntax lets you concisely specify that a function returns some type that implements the `Iterator` trait without needing to write out a very long type.
However, you can only use `impl Trait` if you're returning a single type. For example, this code that returns either a `NewsArticle` or a `SocialPost` with the return type specified as `impl Summary` wouldn't work:
Returning either a `NewsArticle` or a `SocialPost` isn't allowed due to restrictions around how the `impl Trait` syntax is implemented in the compiler. We'll cover how to write a function with this behavior in the ["Using Trait Objects to Abstract over Shared Behavior"][trait-objects] section of Chapter 18.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-05-returning-impl-trait/src/lib.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-06-impl-trait-returns-one-type/src/lib.rs:
here}}
```

[rust_052] Rust - Generics & Traits

Traits: Defining Shared Behavior (impl Trait, Trait Bounds, where clauses): Using Trait Bounds to Conditionally Implement Methods

Sumber: ht

Using Trait Bounds to Conditionally Implement Methods
By using a trait bound with an `impl` block that uses generic type parameters, we can implement methods conditionally for types that implement the specified traits. For example, the type `Pair` in Listing 10-15 always implements the `new` function to return a new instance of `Pair` (recall from the ["Method Syntax"][methods] section of Chapter 5 that `Self` is a type alias for the type of the `impl` block, which in this case is `Pair`). But in the next `impl` block, `Pair` only implements the `cmp_display` method if its inner type `T` implements the `PartialOrd` trait that enables comparison _and_ the `Display` trait that enables printing.
We can also conditionally implement a trait for any type that implements another trait. Implementations of a trait on any type that satisfies the trait bounds are called _blanket implementations_ and are used extensively in the Rust standard library. For example, the standard library implements the `ToString` trait on any type that implements the `Display` trait. The `impl` block in the standard library looks similar to this code:
Because the standard library has this blanket implementation, we can call the `to_string` method defined by the `ToString` trait on any type that implements the `Display` trait. For example, we can turn integers into their corresponding `String` values like this because integers implement `Display`:
Blanket implementations appear in the documentation for the trait in the "Implementors" section.
Traits and trait bounds let us write code that uses generic type parameters to reduce duplication but also specify to the compiler that we want the generic type to have particular behavior. The compiler can then use the trait bound information to check that all

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-15/src/lib.rs}}

// ----------------------------------------

impl<T: Display> ToString for T {
    // --snip--
}

// ----------------------------------------

let s = 3.to_string();
```

[rust_053] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Validating References with Lifetimes

Sumber: ht

Validating References with Lifetimes
Lifetimes are another kind of generic that we've already been using. Rather than ensuring that a type has the behavior we want, lifetimes ensure that references are valid as long as we need them to be.
One detail we didn't discuss in the ["References and Borrowing"][references-and-borrowing] section in Chapter 4 is that every reference in Rust has a lifetime, which is the scope for which that reference is valid. Most of the time, lifetimes are implicit and inferred, just like most of the time, types are inferred. We are only required to annotate types when multiple types are possible. In a similar way, we must annotate lifetimes when the lifetimes of references could be related in a few different ways. Rust requires us to annotate the relationships using generic lifetime parameters to ensure that the actual references used at runtime will definitely be valid.
Annotating lifetimes is not even a concept most other programming languages have, so this is going to feel unfamiliar. Although we won't cover lifetimes in their entirety in this chapter, we'll discuss common ways you might encounter lifetime syntax so that you can get comfortable with the concept.

[rust_054] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Dangling References

Sumber: ht

Dangling References
The main aim of lifetimes is to prevent dangling references, which, if they were allowed to exist, would cause a program to reference data other than the data it's intended to reference. Consider the program in Listing 10-16, which has an outer scope and an inner scope.
> Note: The examples in Listings 10-16, 10-17, and 10-23 declare variables
> without giving them an initial value, so the variable name exists in the outer
> scope. At first glance, this might appear to be in conflict with Rust having
> no null values. However, if we try to use a variable before giving it a value,
> we'll get a compile-time error, which shows that indeed Rust does not allow
> null values.
The outer scope declares a variable named `r` with no initial value, and the inner scope declares a variable named `x` with the initial value of `5`. Inside the inner scope, we attempt to set the value of `r` as a reference to `x`.
Then, the inner scope ends, and we attempt to print the value in `r`. This code won't compile, because the value that `r` is referring to has gone out of scope before we try to use it. Here is the error message:
The error message says that the variable `x` "does not live long enough." The reason is that `x` will be out of scope when the inner scope ends on line 7. But `r` is still valid for the outer scope; because its scope is larger, we say that it "lives longer." If Rust allowed this code to work, `r` would be referencing memory that was deallocated when `x` went out of scope, and anything we tried to do with `r` wouldn't work correctly. So, how does Rust

determine that this code is invalid? It uses a borrow checker.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-16/src/main.rs}}
```

[rust_055] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): The Borrow Checker

Sumber: ht

The Borrow Checker

The Rust compiler has a _borrow checker_ that compares scopes to determine whether all borrows are valid. Listing 10-17 shows the same code as Listing 10-16 but with annotations showing the lifetimes of the variables.

Here, we've annotated the lifetime of `r` with `'a` and the lifetime of `x` with `'b`. As you can see, the inner `'b` block is much smaller than the outer `'a` lifetime block. At compile time, Rust compares the size of the two lifetimes and sees that `r` has a lifetime of `'a` but that it refers to memory with a lifetime of `'b`. The program is rejected because `'b` is shorter than `'a`: The subject of the reference doesn't live as long as the reference.

Listing 10-18 fixes the code so that it doesn't have a dangling reference and it compiles without any errors.

Here, `x` has the lifetime `'b`, which in this case is larger than `'a`. This means `r` can reference `x` because Rust knows that the reference in `r` will always be valid while `x` is valid.

Now that you know where the lifetimes of references are and how Rust analyzes lifetimes to ensure that references will always be valid, let's explore generic lifetimes in function parameters and return values.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-17/src/main.rs}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-18/src/main.rs}}
```

[rust_056] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Generic Lifetimes in Functions

Sumber: ht

Generic Lifetimes in Functions

We'll write a function that returns the longer of two string slices. This function will take two string slices and return a single string slice. After we've implemented the `longest` function, the code in Listing 10-19 should print `The longest string is abcd`.

Note that we want the function to take string slices, which are references, rather than strings, because we don't want the `longest` function to take ownership of its parameters. Refer to ["String Slices as Parameters"][string-slices-as-parameters] in Chapter 4 for more discussion about why the parameters we use in Listing 10-19 are the ones we want.

If we try to implement the `longest` function as shown in Listing 10-20, it won't compile.

Instead, we get the following error that talks about lifetimes:
The help text reveals that the return type needs a generic lifetime parameter on it because Rust can't tell whether the reference being returned refers to `x` or `y`. Actually, we don't know either, because the `if` block in the body of this function returns a reference to `x` and the `else` block returns a reference to `y`!
When we're defining this function, we don't know the concrete values that will be passed into this function, so we don't know whether the `if` case or the `else` case will execute. We also don't know the concrete lifetimes of the references that will be passed in, so we can't look at the scopes as we did in Listings 10-17 and 10-18 to determine whether the reference we return will always be valid. The borrow checker can't determine this either, because it doesn't know how the lifetimes of `x` and `y` relate to the lifetime of the return value. To fix this error, we'll add generic lifetime parameters that define the relationship between the references so that t

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-19/src/main.rs}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-20/src/main.rs:here}}
```

[rust_057] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Lifetime Annotation Syntax

Sumber: ht

Lifetime Annotation Syntax
Lifetime annotations don't change how long any of the references live. Rather, they describe the relationships of the lifetimes of multiple references to each other without affecting the lifetimes. Just as functions can accept any type when the signature specifies a generic type parameter, functions can accept references with any lifetime by specifying a generic lifetime parameter.
Lifetime annotations have a slightly unusual syntax: The names of lifetime parameters must start with an apostrophe (`'`) and are usually all lowercase and very short, like generic types. Most people use the name `'a` for the first lifetime annotation. We place lifetime parameter annotations after the `&` of a reference, using a space to separate the annotation from the reference's type.
Here are some examples -- a reference to an `i32` without a lifetime parameter, a reference to an `i32` that has a lifetime parameter named `'a`, and a mutable reference to an `i32` that also has the lifetime `'a`:
One lifetime annotation by itself doesn't have much meaning, because the annotations are meant to tell Rust how generic lifetime parameters of multiple references relate to each other. Let's examine how the lifetime annotations relate to each other in the context of the `longest` function.

Contoh Kode / Perintah CLI:

```
&i32        // a reference
&'a i32     // a reference with an explicit lifetime
&'a mut i32 // a mutable reference with an explicit lifetime
```

[rust_058] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): In Function Signatures

Sumber: ht

In Function Signatures
To use lifetime annotations in function signatures, we need to declare the
generic lifetime parameters inside angle brackets between the function name and
the parameter list, just as we did with generic type parameters.
We want the signature to express the following constraint: The returned
reference will be valid as long as both of the parameters are valid. This is
the relationship between lifetimes of the parameters and the return value.
We'll name the lifetime `'a` and then add it to each reference, as shown in
Listing 10-21.
This code should compile and produce the result we want when we use it with the
`main` function in Listing 10-19.
The function signature now tells Rust that for some lifetime `'a`, the function
takes two parameters, both of which are string slices that live at least as
long as lifetime `'a`. The function signature also tells Rust that the string
slice returned from the function will live at least as long as lifetime `'a`.
In practice, it means that the lifetime of the reference returned by the
`longest` function is the same as the smaller of the lifetimes of the values
referred to by the function arguments. These relationships are what we want
Rust to use when analyzing this code.
Remember, when we specify the lifetime parameters in this function signature,
we're not changing the lifetimes of any values passed in or returned. Rather,
we're specifying that the borrow checker should reject any values that don't
adhere to these constraints. Note that the `longest` function doesn't need to
know exactly how long `x` and `y` will live, only that some scope can be
substituted for `'a` that will satisfy this signature.
When annotating lifetimes in functions, the annotations go in the function
signature, not in the function bod

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-21/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-22/src/main.rs:here}}

// ----------------------------------------

{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-23/src/main.rs:here}}
```

[rust_059] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Relationships

Sumber: ht

Relationships
The way in which you need to specify lifetime parameters depends on what your
function is doing. For example, if we changed the implementation of the
`longest` function to always return the first parameter rather than the longest
string slice, we wouldn't need to specify a lifetime on the `y` parameter. The

following code will compile:
We've specified a lifetime parameter `'a` for the parameter `x` and the return type, but not for the parameter `y`, because the lifetime of `y` does not have any relationship with the lifetime of `x` or the return value.
When returning a reference from a function, the lifetime parameter for the return type needs to match the lifetime parameter for one of the parameters. If the reference returned does _not_ refer to one of the parameters, it must refer to a value created within this function. However, this would be a dangling reference because the value will go out of scope at the end of the function.
Consider this attempted implementation of the `longest` function that won't compile:
Here, even though we've specified a lifetime parameter `'a` for the return type, this implementation will fail to compile because the return value lifetime is not related to the lifetime of the parameters at all. Here is the error message we get:
The problem is that `result` goes out of scope and gets cleaned up at the end of the `longest` function. We're also trying to return a reference to `result` from the function. There is no way we can specify lifetime parameters that would change the dangling reference, and Rust won't let us create a dangling reference. In this case, the best fix would be to return an owned data type rather than a reference so that the calling function is then responsible for cleaning up the value.
Ultimately, lifetim

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-08-only-one-reference-with-lifetime/src/ma
in.rs:here}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-09-unrelated-lifetime/src/main.rs:here}}
```

[rust_060] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): In Struct Definitions

Sumber: ht

In Struct Definitions
So far, the structs we've defined all hold owned types. We can define structs to hold references, but in that case, we would need to add a lifetime annotation on every reference in the struct's definition. Listing 10-24 has a struct named `ImportantExcerpt` that holds a string slice.
This struct has the single field `part` that holds a string slice, which is a reference. As with generic data types, we declare the name of the generic lifetime parameter inside angle brackets after the name of the struct so that we can use the lifetime parameter in the body of the struct definition. This annotation means an instance of `ImportantExcerpt` can't outlive the reference it holds in its `part` field.
The `main` function here creates an instance of the `ImportantExcerpt` struct that holds a reference to the first sentence of the `String` owned by the variable `novel`. The data in `novel` exists before the `ImportantExcerpt` instance is created. In addition, `novel` doesn't go out of scope until after

the `ImportantExcerpt` goes out of scope, so the reference in the
`ImportantExcerpt` instance is valid.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-24/src/main.rs}}
```

[rust_061] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Lifetime Elision

Sumber: ht

Lifetime Elision
You've learned that every reference has a lifetime and that you need to specify
lifetime parameters for functions or structs that use references. However, we
had a function in Listing 4-9, shown again in Listing 10-25, that compiled
without lifetime annotations.
The reason this function compiles without lifetime annotations is historical:
In early versions (pre-1.0) of Rust, this code wouldn't have compiled, because
every reference needed an explicit lifetime. At that time, the function
signature would have been written like this:
After writing a lot of Rust code, the Rust team found that Rust programmers
were entering the same lifetime annotations over and over in particular
situations. These situations were predictable and followed a few deterministic
patterns. The developers programmed these patterns into the compiler's code so
that the borrow checker could infer the lifetimes in these situations and
wouldn't need explicit annotations.
This piece of Rust history is relevant because it's possible that more
deterministic patterns will emerge and be added to the compiler. In the future,
even fewer lifetime annotations might be required.
The patterns programmed into Rust's analysis of references are called the
lifetime elision rules. These aren't rules for programmers to follow; they're
a set of particular cases that the compiler will consider, and if your code
fits these cases, you don't need to write the lifetimes explicitly.
The elision rules don't provide full inference. If there is still ambiguity
about what lifetimes the references have after Rust applies the rules, the
compiler won't guess what the lifetime of the remaining references should be.
Instead of guessing, the compiler will give you an error that you can resolve
by adding the lifet

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch10-generic-types-traits-and-lifetimes/listing-10-25/src/main.rs:here}}

// ----------------------------------------

fn first_word<'a>(s: &'a str) -> &'a str {

// ----------------------------------------

fn first_word(s: &str) -> &str {
```

[rust_062] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): In Method Definitions

Sumber: ht

In Method Definitions
When we implement methods on a struct with lifetimes, we use the same syntax as that of generic type parameters, as shown in Listing 10-11. Where we declare and use the lifetime parameters depends on whether they're related to the struct fields or the method parameters and return values.
Lifetime names for struct fields always need to be declared after the `impl` keyword and then used after the struct's name because those lifetimes are part of the struct's type.
In method signatures inside the `impl` block, references might be tied to the lifetime of references in the struct's fields, or they might be independent. In addition, the lifetime elision rules often make it so that lifetime annotations aren't necessary in method signatures. Let's look at some examples using the struct named `ImportantExcerpt` that we defined in Listing 10-24.
First, we'll use a method named `level` whose only parameter is a reference to `self` and whose return value is an `i32`, which is not a reference to anything:
The lifetime parameter declaration after `impl` and its use after the type name are required, but because of the first elision rule, we're not required to annotate the lifetime of the reference to `self`.
Here is an example where the third lifetime elision rule applies:
There are two input lifetimes, so Rust applies the first lifetime elision rule and gives both `&self` and `announcement` their own lifetimes. Then, because one of the parameters is `&self`, the return type gets the lifetime of `&self`, and all lifetimes have been accounted for.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-10-lifetimes-on-methods/src/main.rs:1st}}

// ----------------------------------------

{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-10-lifetimes-on-methods/src/main.rs:3rd}}
```

[rust_063] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): The Static Lifetime

Sumber: ht

The Static Lifetime
One special lifetime we need to discuss is `'static`, which denotes that the affected reference _can_ live for the entire duration of the program. All string literals have the `'static` lifetime, which we can annotate as follows:
The text of this string is stored directly in the program's binary, which is always available. Therefore, the lifetime of all string literals is `'static`.
You might see suggestions in error messages to use the `'static` lifetime. But before specifying `'static` as the lifetime for a reference, think about whether or not the reference you have actually lives the entire lifetime of your program, and whether you want it to. Most of the time, an error message suggesting the `'static` lifetime results from attempting to create a dangling

reference or a mismatch of the available lifetimes. In such cases, the solution is to fix those problems, not to specify the `'static` lifetime.

Contoh Kode / Perintah CLI:

```
let s: &'static str = "I have a static lifetime.";
```

[rust_064] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Generic Type Parameters, Trait Bounds, and Lifetimes

Sumber: ht

Generic Type Parameters, Trait Bounds, and Lifetimes
Let's briefly look at the syntax of specifying generic type parameters, trait bounds, and lifetimes all in one function!
This is the `longest` function from Listing 10-21 that returns the longer of two string slices. But now it has an extra parameter named `ann` of the generic type `T`, which can be filled in by any type that implements the `Display` trait as specified by the `where` clause. This extra parameter will be printed using `{}`, which is why the `Display` trait bound is necessary. Because lifetimes are a type of generic, the declarations of the lifetime parameter `'a` and the generic type parameter `T` go in the same list inside the angle brackets after the function name.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include
../listings/ch10-generic-types-traits-and-lifetimes/no-listing-11-generics-traits-and-lifetimes/src/main.
rs:here}}
```

[rust_065] Rust - Lifetimes Validation

Validating References with Lifetimes ('a Lifetime Annotations, Borrow Checker): Summary

Sumber: ht

Summary
We covered a lot in this chapter! Now that you know about generic type parameters, traits and trait bounds, and generic lifetime parameters, you're ready to write code without repetition that works in many different situations. Generic type parameters let you apply the code to different types. Traits and trait bounds ensure that even though the types are generic, they'll have the behavior the code needs. You learned how to use lifetime annotations to ensure that this flexible code won't have any dangling references. And all of this analysis happens at compile time, which doesn't affect runtime performance!
Believe it or not, there is much more to learn on the topics we discussed in this chapter: Chapter 18 discusses trait objects, which are another way to use traits. There are also more complex scenarios involving lifetime annotations that you will only need in very advanced scenarios; for those, you should read the [Rust Reference][reference]. But next, you'll learn how to write tests in Rust so that you can make sure your code is working the way it should.
[references-and-borrowing]: ch04-02-references-and-borrowing.html#references-and-borrowing
[string-slices-as-parameters]: ch04-03-slices.html#string-slices-as-parameters
[reference]: ../reference/trait-bounds.html

[rust_066] Rust - Smart Pointers (Box, Rc, Arc)

Using Box<T> to Point to Data on the Heap (Deref and Drop Traits): Using `Box<T>` to Point to Data on the Heap

Sumber: ht

Using `Box` to Point to Data on the Heap
The most straightforward smart pointer is a box, whose type is written `Box`. _Boxes_ allow you to store data on the heap rather than the stack. What remains on the stack is the pointer to the heap data. Refer to Chapter 4 to review the difference between the stack and the heap.
Boxes don't have performance overhead, other than storing their data on the heap instead of on the stack. But they don't have many extra capabilities either. You'll use them most often in these situations:
- When you have a type whose size can't be known at compile time, and you want to use a value of that type in a context that requires an exact size
- When you have a large amount of data, and you want to transfer ownership but ensure that the data won't be copied when you do so
- When you want to own a value, and you care only that it's a type that implements a particular trait rather than being of a specific type
We'll demonstrate the first situation in "Enabling Recursive Types with Boxes". In the second
case, transferring ownership of a large amount of data can take a long time because the data is copied around on the stack. To improve performance in this situation, we can store the large amount of data on the heap in a box. Then, only the small amount of pointer data is copied around on the stack, while the data it references stays in one place on the heap. The third case is known as a _trait object_, and ["Using Trait Objects to Abstract over Shared Behavior"][trait-objects] in Chapter 18 is devoted to that
topic. So, what you learn here you'll apply again in that section!

[rust_067] Rust - Smart Pointers (Box, Rc, Arc)

Using Box<T> to Point to Data on the Heap (Deref and Drop Traits): Storing Data on the Heap

Sumber: ht

Storing Data on the Heap
Before we discuss the heap storage use case for `Box`, we'll cover the syntax and how to interact with values stored within a `Box`.
Listing 15-1 shows how to use a box to store an `i32` value on the heap. We define the variable `b` to have the value of a `Box` that points to the value `5`, which is allocated on the heap. This program will print `b = 5`; in this case, we can access the data in the box similarly to how we would if this data were on the stack. Just like any owned value, when a box goes out of scope, as `b` does at the end of `main`, it will be deallocated. The deallocation happens both for the box (stored on the stack) and the data it points to (stored on the heap).
Putting a single value on the heap isn't very useful, so you won't use boxes by themselves in this way very often. Having values like a single `i32` on the stack, where they're stored by default, is more appropriate in the majority of situations. Let's look at a case where boxes allow us to define types that we wouldn't be allowed to define if we didn't have boxes.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch15-smart-pointers/listing-15-01/src/main.rs}}
```

[rust_068] Rust - Smart Pointers (Box, Rc, Arc)

Using Box<T> to Point to Data on the Heap (Deref and Drop Traits): Enabling Recursive Types with Boxes

Sumber: ht

Enabling Recursive Types with Boxes
A value of a _recursive type_ can have another value of the same type as part of itself. Recursive types pose an issue because Rust needs to know at compile time how much space a type takes up. However, the nesting of values of recursive types could theoretically continue infinitely, so Rust can't know how much space the value needs. Because boxes have a known size, we can enable recursive types by inserting a box in the recursive type definition.
As an example of a recursive type, let's explore the cons list. This is a data type commonly found in functional programming languages. The cons list type we'll define is straightforward except for the recursion; therefore, the concepts in the example we'll work with will be useful anytime you get into more complex situations involving recursive types.
Understanding the Cons List
A _cons list_ is a data structure that comes from the Lisp programming language and its dialects, is made up of nested pairs, and is the Lisp version of a linked list. Its name comes from the `cons` function (short for _construct function_) in Lisp that constructs a new pair from its two arguments. By calling `cons` on a pair consisting of a value and another pair, we can construct cons lists made up of recursive pairs.
For example, here's a pseudocode representation of a cons list containing the list `1, 2, 3` with each pair in parentheses:
Each item in a cons list contains two elements: the value of the current item and of the next item. The last item in the list contains only a value called `Nil` without a next item. A cons list is produced by recursively calling the `cons` function. The canonical name to denote the base case of the recursion is `Nil`. Note that this is not the same as the "null" or "nil

Contoh Kode / Perintah CLI:

```
Each item in a cons list contains two elements: the value of the current item
and of the next item. The last item in the list contains only a value called
`Nil` without a next item. A cons list is produced by recursively calling the
`cons` function. The canonical name to denote the base case of the recursion is
`Nil`. Note that this is not the same as the "null" or "nil" concept discussed
in Chapter 6, which is an invalid or absent value.

The cons list isn't a commonly used data structure in Rust. Most of the time
when you have a list of items in Rust, `Vec<T>` is a better choice to use.
Other, more complex recursive data types _are_ useful in various situations,
but by starting with the cons list in this chapter, we can explore how boxes
let us define a recursive data type without much distraction.

Listing 15-2 contains an enum definition for a cons list. Note that this code
won't compile yet, because the `List` type doesn't have a known size, which
we'll demonstrate.

<Listing number="15-2" file-name="src/main.rs" caption="The first attempt at defining an enum to
represent a cons list data structure of `i32` values">

// ----------------------------------------
```

```
</Listing>

> Note: We're implementing a cons list that holds only `i32` values for the
> purposes of this example. We could have implemented it using generics, as we
> discussed in Chapter 10, to define a cons list type that could store values of
> any type.

Using the `List` type to store the list `1, 2, 3` would look like the code in
Listing 15-3.

<Listing number="15-3" file-name="src/main.rs" caption="Using the `List` enum to store the list `1, 2,
3`">

// ----------------------------------------

</Listing>

The first `Cons` value holds `1` and another `List` value. This `List` value is
another `Cons` value that holds `2` and another `List` value. This `List` value
is one more `Cons` value that holds `3` and a `List` value, which is finally
`Nil`, the non-recursive variant that signals the end of the list.

If we try to compile the code in Listing 15-3, we get the error shown in
Listing 15-4.

<Listing number="15-4" caption="The error we get when attempting to define a recursive enum">
```

[rust_069] Rust - Fearless Concurrency (Threads & Channels)

Fearless Concurrency (spawn threads, move closures, message passing): Using Threads to Run Code Simultaneously

Sumber: ht

Using Threads to Run Code Simultaneously

In most current operating systems, an executed program's code is run in a _process_, and the operating system will manage multiple processes at once. Within a program, you can also have independent parts that run simultaneously. The features that run these independent parts are called _threads_. For example, a web server could have multiple threads so that it can respond to more than one request at the same time.

Splitting the computation in your program into multiple threads to run multiple tasks at the same time can improve performance, but it also adds complexity. Because threads can run simultaneously, there's no inherent guarantee about the order in which parts of your code on different threads will run. This can lead to problems, such as:

- Race conditions, in which threads are accessing data or resources in an inconsistent order
- Deadlocks, in which two threads are waiting for each other, preventing both threads from continuing
- Bugs that only happen in certain situations and are hard to reproduce and fix reliably

Rust attempts to mitigate the negative effects of using threads, but programming in a multithreaded context still takes careful thought and requires a code structure that is different from that in programs running in a single thread.

Programming languages implement threads in a few different ways, and many

operating systems provide an API the programming language can call for creating new threads. The Rust standard library uses a _1:1_ model of thread implementation, whereby a program uses one operating system thread per one language thread. There are crates that implement other models of threading that make different trade-offs to the 1:1 model. (Rust's async system, which we will see in the next chapter, p

[rust_070] Rust - Fearless Concurrency (Threads & Channels)

Fearless Concurrency (spawn threads, move closures, message passing): Creating a New Thread with `spawn`

Sumber: ht

Creating a New Thread with `spawn`

To create a new thread, we call the `thread::spawn` function and pass it a closure (we talked about closures in Chapter 13) containing the code we want to run in the new thread. The example in Listing 16-1 prints some text from a main thread and other text from a new thread.

Note that when the main thread of a Rust program completes, all spawned threads are shut down, whether or not they have finished running. The output from this program might be a little different every time, but it will look similar to the following:

The calls to `thread::sleep` force a thread to stop its execution for a short duration, allowing a different thread to run. The threads will probably take turns, but that isn't guaranteed: It depends on how your operating system schedules the threads. In this run, the main thread printed first, even though the print statement from the spawned thread appears first in the code. And even though we told the spawned thread to print until `i` is `9`, it only got to `5` before the main thread shut down.

If you run this code and only see output from the main thread, or don't see any overlap, try increasing the numbers in the ranges to create more opportunities for the operating system to switch between the threads.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch16-fearless-concurrency/listing-16-01/src/main.rs}}
```

[rust_071] Rust - Fearless Concurrency (Threads & Channels)

Fearless Concurrency (spawn threads, move closures, message passing): Waiting for All Threads to Finish

Sumber: ht

Waiting for All Threads to Finish

The code in Listing 16-1 not only stops the spawned thread prematurely most of the time due to the main thread ending, but because there is no guarantee on the order in which threads run, we also can't guarantee that the spawned thread will get to run at all!

We can fix the problem of the spawned thread not running or of it ending prematurely by saving the return value of `thread::spawn` in a variable. The return type of `thread::spawn` is `JoinHandle`. A `JoinHandle` is an owned value that, when we call the `join` method on it, will wait for its thread to finish. Listing 16-2 shows how to use the `JoinHandle` of the thread we created in Listing 16-1 and how to call `join` to make sure the spawned thread finishes before `main` exits.

` from `thread::spawn` to guarantee the thread is run to completion">

Calling `join` on the handle blocks the thread currently running until the thread represented by the handle terminates. _Blocking_ a thread means that thread is prevented from performing work or exiting. Because we've put the call to `join` after the main thread's `for` loop, running Listing 16-2 should produce output similar to this:
The two threads continue alternating, but the main thread waits because of the call to `handle.join()` and does not end until the spawned thread is finished.
But let's see what happens when we instead move `handle.join()` before the `for` loop in `main`, like this:
The main thread will wait for the spawned thread to finish and then run its `for` loop, so the output won't be interleaved anymore, as shown here:
Small details, such as where `join` is called, can affect whether or not your threads run at the same time.

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch16-fearless-concurrency/listing-16-02/src/main.rs}}

// ----------------------------------------

The two threads continue alternating, but the main thread waits because of the
call to `handle.join()` and does not end until the spawned thread is finished.

But let's see what happens when we instead move `handle.join()` before the
`for` loop in `main`, like this:

<Listing file-name="src/main.rs">

// ----------------------------------------

</Listing>

The main thread will wait for the spawned thread to finish and then run its
`for` loop, so the output won't be interleaved anymore, as shown here:

<!-- Not extracting output because changes to this output aren't significant;
the changes are likely to be due to the threads running differently rather than
changes in the compiler -->
```

[rust_072] Rust - Fearless Concurrency (Threads & Channels)

Fearless Concurrency (spawn threads, move closures, message passing): Using `move` Closures with Threads

Sumber: ht

Using `move` Closures with Threads
We'll often use the `move` keyword with closures passed to `thread::spawn` because the closure will then take ownership of the values it uses from the environment, thus transferring ownership of those values from one thread to another. In ["Capturing References or Moving Ownership"][capture] in Chapter 13, we discussed `move` in the context of closures. Now we'll
concentrate more on the interaction between `move` and `thread::spawn`.
Notice in Listing 16-1 that the closure we pass to `thread::spawn` takes no arguments: We're not using any data from the main thread in the spawned thread's code. To use data from the main thread in the spawned thread, the spawned thread's closure must capture the values it needs. Listing 16-3 shows an attempt to create a vector in the main thread and use it in the spawned thread. However, this won't work yet, as you'll see in a moment.

The closure uses `v`, so it will capture `v` and make it part of the closure's environment. Because `thread::spawn` runs this closure in a new thread, we should be able to access `v` inside that new thread. But when we compile this example, we get the following error:
Rust _infers_ how to capture `v`, and because `println!` only needs a reference to `v`, the closure tries to borrow `v`. However, there's a problem: Rust can't tell how long the spawned thread will run, so it doesn't know whether the reference to `v` will always be valid.
Listing 16-4 provides a scenario that's more likely to have a reference to `v` that won't be valid.
If Rust allowed us to run this code, there's a possibility that the spawned thread would be immediately put in the background without running at all. The spawned thread has a reference to `v` inside, but the main thread immediately drops `v`

Contoh Kode / Perintah CLI:

```
{{#rustdoc_include ../listings/ch16-fearless-concurrency/listing-16-03/src/main.rs}}

// ----------------------------------------

Rust _infers_ how to capture `v`, and because `println!` only needs a reference
to `v`, the closure tries to borrow `v`. However, there's a problem: Rust can't
tell how long the spawned thread will run, so it doesn't know whether the
reference to `v` will always be valid.

Listing 16-4 provides a scenario that's more likely to have a reference to `v`
that won't be valid.

<Listing number="16-4" file-name="src/main.rs" caption="A thread with a closure that attempts to capture
a reference to `v` from a main thread that drops `v`">

// ----------------------------------------

</Listing>

If Rust allowed us to run this code, there's a possibility that the spawned
thread would be immediately put in the background without running at all. The
spawned thread has a reference to `v` inside, but the main thread immediately
drops `v`, using the `drop` function we discussed in Chapter 15. Then, when the
spawned thread starts to execute, `v` is no longer valid, so a reference to it
is also invalid. Oh no!

To fix the compiler error in Listing 16-3, we can use the error message's
advice:

<!-- manual-regeneration
after automatic regeneration, look at listings/ch16-fearless-concurrency/listing-16-03/output.txt and
copy the relevant part
-->
```